Angular — Zero to Master

36 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (10 topics)

01 Introduction to Angular

02 Environment Setup and Angular CLI

03 TypeScript Essentials

04 Components

05 Data Binding

06 Control Flow (@if, @for, @switch)

07 Directives, Class and Style Binding

08 Signals (signal, computed, effect)

09 Component Communication (input, output, model)

10 Pipes

Intermediate (12 topics)

11 Services and Dependency Injection

12 Template-Driven Forms

13 Reactive Forms

14 Form Validation and Custom Validators

15 Routing Basics

16 Route Parameters, Query Params and Child Routes

17 Route Guards and Lazy Loading

18 HttpClient and Working with APIs

19 RxJS Fundamentals

20 Component Lifecycle Hooks

21 Content Projection and ViewChild

22 Custom Directives

Advanced (9 topics)

23	HTTP Interceptors and Error Handling
24	Advanced RxJS Operators
25	State Management (Signals, Services, NgRx)
26	Change Detection and OnPush
27	Advanced Dependency Injection
28	Deferrable Views and Performance
29	Testing Angular Apps
30	SSR, Hydration and SEO
31	Animations and UI Libraries

Master (5 topics)

32	Real Project 1: Task Manager
33	Real Project 2: Product Catalog with API
34	Architecture and Best Practices
35	Security and Accessibility
36	Build, Deploy and CI/CD

Beginner Level

01. Introduction to Angular

DEFINITION

Angular is a free, open-source framework from Google for building web applications using **TypeScript**, HTML and CSS. Instead of writing one huge page, you build small reusable pieces called **components** and snap them together like LEGO blocks.

Angular apps are usually **Single Page Applications (SPA)**: the browser loads the page once, and after that Angular only swaps the parts of the screen that change. No full page reload, so the app feels fast like a mobile app.

Angular is a complete platform: it ships with routing, forms, HTTP, testing tools and a command-line tool (the Angular CLI), so you do not have to pick and glue together many separate libraries.

SYNTAX

```
@Component({
  selector: 'app-name',        // the HTML tag you will use
  template: `<h1>...</h1>`,    // what to show (or templateUrl: './x.html')
  styles: `h1 { color: red }`, // how it looks (or styleUrl)
})
export class NameComponent {
  // data and methods for the template
}
```

EXAMPLE

```
import { Component } from '@angular/core';

// A component is a TypeScript class + an HTML template + CSS styles.
@Component({
  selector: 'app-root', // the tag <app-root> in index.html will show this component
  template: `
    <h1>{{ title }}</h1>
    <p>{{ message }}</p>
  `,
  styles: `
    h1 { color: #dd0031; }
  `,
})
export class HelloWorldComponent {
  // Properties of the class can be shown in the template with {{ }}
  title = 'Hello, Angular!';
  message = 'My first Angular component is running.';
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Think of a railway station display board: only the train timings change on the screen, the whole building is not rebuilt. An Angular single page app refreshes only the part of the page that changed.

தமிழ்

சென்னை மெட்ரோ ரயில் நிலையத்தில் உள்ள அறிவிப்புத் திரையை நினைத்துப் பாருங்கள். அடுத்த ரயில் எப்போது வரும் என்ற நேரம் மட்டும் மாறுகிறது; முழு நிலையக் கட்டிடமும் மீண்டும் கட்டப்படுவதில்லை. அதே போலத்தான் Angular-இல் (SPA) பக்கம் ஒரு முறை மட்டுமே ஏற்றப்படும், அதன் பிறகு மாற வேண்டிய பகுதி மட்டும் புதுப்பிக்கப்படும்.

அந்தத் திரையில் உள்ள ஒவ்வொரு பகுதியும் - ரயில் பெயர், நேரம், நடைமேடை எண் - ஒவ்வொரு தனித் **component** போல. பல component-களை ஒன்றாகச் சேர்த்துதான் முழு Angular செயலியை உருவாக்குகிறோம்.

PROS

- Complete framework (router, forms, HTTP)
- Strong TypeScript tooling
- Backed by Google, long-term support
- Good for large teams

CONS

- Steeper learning curve than React or Vue
- More boilerplate to learn
- Larger initial bundle for tiny apps

ADVANCED / NEW FEATURES

- Standalone components are the default (v19+)
- Signals for reactivity
- New `@if/@for` control flow
- Zoneless change detection (default in new v21 apps)
- Vite/esbuild based builds

Common mistakes

- Confusing **AngularJS** (old, 2010, JavaScript) with **Angular** (2016+, TypeScript). They are different frameworks.
- Thinking you must learn everything at once. Start with components and templates; the rest comes step by step.
- Skipping TypeScript basics, then getting stuck on error messages.

Interview questions

Q1. What is Angular?
A TypeScript-based, open-source framework by Google for building single page web applications using components.

Q2. What is the difference between AngularJS and Angular?
AngularJS (1.x) is JavaScript with controllers and scopes. Angular (2+) is a TypeScript rewrite built on components, with better performance and tooling.

Q3. What is a Single Page Application?
An app that loads one HTML page and updates parts of it dynamically using JavaScript instead of reloading full pages.

Practice

1. Write down 3 websites you use daily that behave like a SPA (no full reload when you click around).
2. In `hello-world.component.ts`, change the title and message, then add a second `<p>` using a new property.
3. Draw (on paper) the component tree for a shopping website: header, product list, product card, cart.

02. Environment Setup and Angular CLI

DEFINITION

To build Angular apps you need **Node.js** (runs JavaScript tools on your computer), **npm** (installs packages) and the **Angular CLI** (the `ng` command). The CLI creates projects, generates components, starts a dev server and builds your app for production.

A new project contains `src/main.ts` (the entry point that starts the app), `src/app/app.ts` (the root component), `src/app/app.config.ts` (app-wide settings/providers) and `angular.json` (build settings).

SYNTAX

```
# install tools (once)
npm install -g @angular/cli

# create and run a project
ng new my-first-app
cd my-first-app
ng serve            # http://localhost:4200

# generate code
ng generate component product-card   # short: ng g c product-card
ng generate service data
ng build            # production build into dist/
```

EXAMPLE

```
import {
  ApplicationConfig,
  Component,
  provideBrowserGlobalErrorListeners,
} from '@angular/core';

// In a real project this goes in src/app/app.config.ts.
// main.ts then runs:  bootstrapApplication(App, appConfig)
export const appConfig: ApplicationConfig = {
  providers: [provideBrowserGlobalErrorListeners()],
};

// In a real project this is src/app/app.ts (the root component made by `ng new`).
@Component({
  selector: 'app-root',
  template: `
    <h1>Welcome to {{ appName }}</h1>
    <p>Created with: ng new {{ appName }}</p>
    <p>Started with: ng serve</p>
  `,
```

```
})
export class App {
  appName = 'my-first-app';
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Opening a new shop: instead of arranging the building, electricity and shelves yourself, a contractor sets everything up in a day. `ng new` is that contractor.

தமிழ்

ஒரு புதிய கடை திறக்கும்போது கட்டிடம், மின்சாரம், தண்ணீர் இணைப்பு, அலமாரிகள் எல்லாம் முதலில் தயார் செய்ய வேண்டும். அந்த வேலையை ஒவ்வொன்றாகச் செய்யாமல், ஒரு ஒப்பந்தக்காரர் (contractor) ஒரே நாளில் எல்லாவற்றையும் அமைத்துக் கொடுத்தால் எவ்வளவு எளிது!

Angular CLI அந்த ஒப்பந்தக்காரர் போல. `ng new` என்று ஒரு கட்டளை கொடுத்தால் முழு project அமைப்பும் தயார்; `ng generate component` என்றால் புதிய அலமாரி தயார்; `ng serve` என்றால் கடை திறந்து வாடிக்கையாளர் பார்க்கலாம்.

PROS

- One command scaffolds a whole project
- Generators keep a consistent structure
- Built-in dev server, build and test commands

CONS

- Needs Node.js and many npm packages
- First install can be slow

ADVANCED / NEW FEATURES

- Application builder uses esbuild + Vite (very fast builds)
- `ng new` creates standalone apps
- Vitest is the default test runner in v21

Common mistakes

- Using an unsupported Node.js version. Check the Angular docs for the compatible Node versions.
- Running `ng serve` outside the project folder (it must be where `angular.json` is).
- Editing files in `node_modules` or `dist`; they are generated and your changes will be lost.

Interview questions

Q1. What is the Angular CLI?
A command-line tool (`ng`) that creates, builds, tests and serves Angular applications and generates code.

Q2. What does `main.ts` do?
It is the entry point; it calls `bootstrapApplication` to start the root component.

Q3. What is `angular.json`?
The workspace configuration file holding build, serve and test settings for projects.

Practice

1. Install Node.js and the Angular CLI, then run `ng version` and note the versions.
2. Create a project with `ng new demo`, run `ng serve`, and change the title in `app.ts` while the server is running.
3. Run `ng generate component hello` and list which files were created.

03. TypeScript Essentials

DEFINITION

TypeScript is JavaScript with **types**. You tell the language what kind of value a variable holds (`string`, `number`, `boolean`...), and the editor warns you about mistakes before you even run the code.

Angular is written in TypeScript, so you will use **interfaces** (shapes of objects), **classes**, **generics** and **decorators** (like `@Component`) every day. Browsers cannot run TypeScript directly; the Angular CLI compiles it to JavaScript for you.

SYNTAX

```ts
let name: string = 'Angular';
let count: number = 10;
let tags: string[] = ['a', 'b'];

interface User { id: number; name: string; email?: string }  // ? = optional

function add(a: number, b: number): number { return a + b; }

class Student { constructor(public name: string) {} }

function first<T>(items: T[]): T | undefined { return items[0]; }  // generic
```

EXAMPLE

```ts
// Run with: npx tsx types-basics.ts
export {};

// 1. Basic types
const courseName: string = 'Angular';
let lessons: number = 38;
const isFun: boolean = true;
const topics: string[] = ['Components', 'Services', 'Routing'];
const level: [string, number] = ['Beginner', 10]; // tuple: fixed length and types

// 2. Union of literal values: only these three strings are allowed
type Level = 'Beginner' | 'Intermediate' | 'Advanced';

function describe(level: Level, count: number = 1): string {
  return `${count} topic(s) at ${level} level`;
}

// 3. Generic function: works for any type T
function first<T>(items: T[]): T | undefined {
  return items[0];
```

```
}

console.log(`${courseName} has ${lessons} lessons. Fun? ${isFun}`);
console.log('Topics:', topics.join(', '));
console.log('Tuple:', level[0], level[1]);
console.log(describe('Beginner', 10));
console.log('First topic:', first(topics));
console.log('First of empty list:', first<number>([]));

// describe('Expert'); // <- compile error: 'Expert' is not a valid Level
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

At a ration shop, rice is measured in kilograms and oil in litres. If someone asks for 'two litres of rice', the shopkeeper stops the mistake immediately. Types do the same for your code before it runs.

தமிழ்

ரேஷன் கடையில் பொருள் வாங்கும்போது அரிசிக்கு தனி அளவு (கிலோ), எண்ணெய்க்கு தனி அளவு (லிட்டர்) என்று ஒவ்வொன்றுக்கும் ஒரு வகை உண்டு. யாராவது 'இரண்டு லிட்டர் அரிசி' என்று கேட்டால் கடைக்காரர் உடனே 'அது தவறு' என்று சொல்லிவிடுவார்.

TypeScript-உம் அப்படியே. `number`, `string` என்று ஒவ்வொரு மதிப்புக்கும் வகையைச் சொல்லிவிட்டால், தவறான வகையைப் பயன்படுத்தினால் நீங்கள் program-ஐ ஓட்டுவதற்கு முன்பே editor எச்சரிக்கை கொடுக்கும். இதனால் பிழைகள் குறையும்.

PROS

- Catches bugs at compile time
- Great autocomplete and refactoring
- Self-documenting code

CONS

- Extra compile step
- Types can be verbose in complex cases
- Types vanish at runtime

ADVANCED / NEW FEATURES

- `satisfies` operator
- Standard decorators (TS 5)
- `const` type parameters
- Inferred type predicates
- Node.js can run simple TypeScript directly (type stripping)

Common mistakes

- Using `any` everywhere. It switches off type checking and defeats the purpose of TypeScript.

- Forgetting that types disappear at runtime; you cannot check an interface with `typeof` or `instanceof`.
- Mixing `==` and `===`; always prefer `===`.

Interview questions

Q1. What is TypeScript and why does Angular use it?
A typed superset of JavaScript. Types catch bugs early, give great editor support and make large apps easier to maintain.

Q2. Interface vs type alias?
Both describe shapes. Interfaces can be extended/merged and suit object shapes; type aliases can also name unions, tuples and primitives.

Q3. What is `any` vs `unknown`?
`any` turns off checks. `unknown` is a safe version: you must narrow the type before using the value.

Practice

1. Create an interface `Product` (id, name, price, inStock?) and an array of three products; print the names of those in stock.
2. Write a generic function `last<T>(items: T[])` that returns the last item, and test it with numbers and strings.
3. Make a class `BankAccount` with a private balance, `deposit`, `withdraw` (no negative balance) and a `balance` getter.

04. Components

DEFINITION

A **component** controls one part of the screen. It has three parts: a **class** (data and behaviour), a **template** (HTML) and **styles** (CSS). The `@Component` decorator ties them together and gives the component a **selector**, the custom HTML tag you write to use it.

To use one component inside another, add it to the parent's `imports` array and write its selector in the parent's template. Styles written in a component are **scoped**: they only affect that component.

SYNTAX

```
@Component({
  selector: 'app-child',
  imports: [OtherComponent],     // components/pipes/directives used in the template
  templateUrl: './child.html',   // or inline: template: `...`
  styleUrl: './child.css',       // or inline: styles: `...`
})
export class ChildComponent {}
```

EXAMPLE

```
import { Component } from '@angular/core';

// A small reusable child component
@Component({
  selector: 'app-profile-card',
  template: `
    <div class="card">
      <h3>{{ name }}</h3>
      <p>{{ role }} - {{ city }}</p>
```

```
    </div>
  `,
  // Styles are scoped: they only apply inside this component
  styles: `
    .card { border: 1px solid #999; padding: 8px; margin: 6px; border-radius: 6px; }
    h3 { margin: 0; }
  `,
})
export class ProfileCardComponent {
  name = 'Priya';
  role = 'Frontend Developer';
  city = 'Coimbatore';
}

// The parent (root) component uses the child by its selector
@Component({
  selector: 'app-root',
  imports: [ProfileCardComponent], // <- must list every component used in the template
  template: `
    <h1>Our Team</h1>
    <app-profile-card />
    <app-profile-card />
  `,
})
export class AppComponent {}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A wedding hall has separate teams for catering, decoration and reception; each handles only its own job. Each Angular component handles one part of the screen.

தமிழ்

ஒரு திருமண மண்டபத்தை எடுத்துக்கொள்ளுங்கள். சமையல் பகுதி, அலங்காரம், வரவேற்பு, உணவு பரிமாறும் பகுதி என்று ஒவ்வொன்றுக்கும் தனிப் பொறுப்பாளர் இருப்பார். ஒவ்வொருவரும் தன் வேலையை மட்டும் பார்த்துக்கொள்வார்கள்; எல்லாம் சேர்ந்துதான் திருமணம் நடக்கும்.

Angular-இல் ஒவ்வொரு **component**-உம் அப்படியே ஒரு பொறுப்பாளர். Header, Footer, Product list என்று தனித்தனியாக எழுதி, `imports`-இல் சேர்த்து, பெரிய component-இல் பயன்படுத்துகிறோம். ஒரு பகுதியில் மாற்றம் செய்தால் மற்றவை பாதிக்கப்படாது - styles scoped ஆக இருப்பதால்.

PROS

- Reusable building blocks
- Easy to test in isolation
- Scoped styles avoid CSS clashes

CONS

- Too many tiny components can add overhead
- Need to import each component where it is used

ADVANCED / NEW FEATURES

- Standalone by default (no NgModule)
- Self-closing tags like `<app-card />`
- `host` metadata replaces `@HostBinding` / `@HostListener`

Common mistakes

- Forgetting to add the child component to the parent's `imports`; Angular then reports an unknown element.
- Using a selector that does not match the tag in the template (typos like `app-profilecard`).
- Putting too much code in one big component instead of splitting it.

Interview questions

Q1. What is a component?
A class decorated with `@Component` that controls a view through a template and styles.

Q2. What is a selector?
The custom HTML tag (or attribute) used to place a component in a template.

Q3. What does `standalone` mean?
A standalone component declares its own `imports` and needs no NgModule. It is the default since Angular 19.

Practice

1. Make a `product-card` component showing a name and price, and display it three times.
2. Add a header and footer component and compose them in the root component.
3. Move the template and CSS of `profile-card` into separate files using `templateUrl` and `styleUrl`.

05. Data Binding

DEFINITION

Data binding connects the component class to its template so the screen stays in sync with your data. Angular has four kinds: **interpolation** `{{ }}` (show a value), **property binding** `[prop]` (set an element property), **event binding** `(event)` (react to user actions) and **two-way binding** `[(ngModel)]` (both directions at once).

Remember the brackets: `[ ]` sends data *into* the template, `( )` sends events *out* of it, and `[( )]` (banana in a box) does both. For `ngModel` you must import `FormsModule`.

SYNTAX

```
{{ expression }}                 <!-- interpolation -->
<button [disabled]="isBusy">       <!-- property binding -->
<div [attr.aria-label]="label">    <!-- attribute binding -->
<button (click)="save()">          <!-- event binding -->
<input [(ngModel)]="name" />       <!-- two-way (needs FormsModule) -->
```

EXAMPLE

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms'; // needed for [(ngModel)]
```

```ts
@Component({
  selector: 'app-root',
  imports: [FormsModule],
  template: `
    <!-- interpolation: show a class value as text -->
    <h2>{{ brand }}</h2>

    <!-- two-way binding [(ngModel)] + attribute binding [attr.aria-label] -->
    <input [(ngModel)]="name" placeholder="Your name" [attr.aria-label]="'Name for ' + brand"
/>

    <!-- event binding (click) + property binding [disabled] -->
    <button (click)="greet()" [disabled]="name.trim() === ''">Greet</button>

    <p>You typed: {{ name }}</p>
    <p>{{ greeting }}</p>
    <p>Button clicked {{ clicks }} time(s)</p>
  `,
})
export class DataBindingComponent {
  brand = 'Chennai Tea House';
  name = '';
  greeting = '';
  clicks = 0;

  greet() {
    this.greeting = `Vanakkam, ${this.name}!`;
    this.clicks++;
  }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

At a tea stall you tell the master 'less sugar' and read the price board. Information flows both ways: the board shows data to you, and your order goes back to the master.

தமிழ்

தேநீர்க் கடையில் நீங்கள் 'ஒரு டீ, சர்க்கரை குறைவா' என்று சொல்கிறீர்கள். மாஸ்டர் உடனே அதை கவனித்து அதன்படி டீ போடுகிறார்; விலைப்பட்டியல் பலகையில் உள்ள விலையை நீங்கள் படித்துக்கொள்கிறீர்கள். இரண்டு பக்கமும் தகவல் பரிமாறப்படுகிறது.

பலகையில் உள்ள விலை = `{{ price }}` (class-இலிருந்து திரைக்கு). நீங்கள் சொல்லும் ஆர்டர் = `(click)` அல்லது `(input)` (திரையிலிருந்து class-க்கு). இரண்டும் எப்போதும் ஒத்திசைவாக இருப்பது = `[(ngModel)]`.

PROS

- Less manual DOM code
- View always matches data
- Declarative and readable

CONS

- Wrong brackets cause confusing errors

- Heavy logic in templates hurts performance

ADVANCED / NEW FEATURES

- `@let` template variables
- Signal-based `model()` for two-way binding
- Stricter template type checking

Common mistakes

- Forgetting to import `FormsModule`, which causes "Can't bind to 'ngModel'".
- Writing `{{ }}` inside a property binding like `[title]="{{ x }}"`; use `[title]="x"` instead.
- Mixing up `[ ]` and `( )` brackets.

Interview questions

Q1. What are the types of data binding in Angular?
Interpolation, property binding, attribute binding, class/style binding, event binding and two-way binding.

Q2. Interpolation vs property binding?
Interpolation converts the value to a string inside text; property binding sets a DOM property with its real type (boolean, object...).

Q3. What is two-way binding?
Combining property and event binding, written `[(ngModel)]`, so the view and the class update each other.

Practice

1. Build a temperature converter: an input for Celsius and text showing Fahrenheit.
2. Add a button that is disabled until a checkbox is ticked.
3. Make a character counter that shows how many letters are typed in a textarea.

06. Control Flow (@if, @for, @switch)

DEFINITION

Control flow decides *what* appears in the template. Angular's built-in blocks are `@if` / `@else` (conditions), `@for` (repeat a list) and `@switch` (choose one of many cases). They replaced the older `*ngIf` and `*ngFor` and need no imports.

`@for` needs a **track** expression (usually an id) so Angular can reuse DOM elements efficiently when the list changes. Add `@empty` to show a message when the list is empty.

SYNTAX

```
@if (condition) { ... } @else if (other) { ... } @else { ... }

@for (item of items; track item.id; let i = $index) { ... } @empty { ... }

@switch (value) {
  @case ('a') { ... }
  @default { ... }
}
```

EXAMPLE

```ts
import { Component, signal } from '@angular/core';

interface Tea {
  name: string;
  price: number;
  available: boolean;
}

@Component({
  selector: 'app-root',
  template: `
    <button (click)="open.set(!open())">{{ open() ? 'Close' : 'Open' }} shop</button>

    @if (open()) {
      <h3>Today's Menu</h3>
      <ul>
        @for (item of menu; track item.name; let i = $index) {
          <li>
            {{ i + 1 }}. {{ item.name }} - ₹{{ item.price }}
            @switch (item.available) {
              @case (true) { (available) }
              @case (false) { (sold out) }
            }
          </li>
        } @empty {
          <li>No items today</li>
        }
      </ul>
    } @else {
      <p>The shop is closed.</p>
    }
  `,
})
export class TeaMenuComponent {
  open = signal(false);
  menu: Tea[] = [
    { name: 'Filter Coffee', price: 20, available: true },
    { name: 'Masala Tea', price: 15, available: true },
    { name: 'Badam Milk', price: 30, available: false },
  ];
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

At a cinema counter: if tickets are available you sell, else you show 'House Full'; for every person in the queue you do the same steps; if nobody is in the queue you show 'No show'. That is `@if`, `@for`, `@empty`.

தமிழ்

ஒரு திரையரங்கு டிக்கெட் கவுண்டரைப் பாருங்கள். 'டிக்கெட் இருக்கிறதா?' என்பதே `@if`. இருந்தால் டிக்கெட் கொடுப்பார்; இல்லையென்றால் 'ஹவுஸ்ஃபுல்' பலகை காட்டுவார் (`@else`).

வரிசையில் நிற்கும் ஒவ்வொருவருக்கும் ஒரே செயலைச் செய்வது `@for`. யாருமே வரிசையில் இல்லையென்றால் 'இன்று காட்சி இல்லை' என்று காட்டுவது `@empty`.

காலை, மதியம், இரவு என்று காட்சி நேரத்துக்கு ஏற்ப வேறு வேறு வேலை செய்வது `@switch`.

PROS

- No imports needed
- Cleaner syntax than `*ngIf` / `*ngFor`
- Faster (built into the compiler)
- `track` improves list performance

CONS

- `track` is mandatory in `@for`
- Older code must be migrated

ADVANCED / NEW FEATURES

- Stable since v18
- `*ngIf`, `*ngFor` deprecated
- Automatic migration: `ng generate @angular/core:control-flow`

Common mistakes

- Leaving out `track` in `@for` (it is required) or tracking by array index for lists that change.
- Still using `*ngIf` / `*ngFor` in new code; the new blocks are faster and simpler.
- Forgetting that signals are called like functions: `open()` not `open`.

Interview questions

Q1. What replaced `*ngIf` and `*ngFor`?
The built-in control flow blocks `@if`, `@for` and `@switch` (stable since Angular 17).

Q2. Why is `track` required in `@for`?
It lets Angular identify items so it can move/reuse DOM nodes instead of re-creating the whole list.

Q3. What is `@empty`?
A block displayed when the `@for` collection has no items.

Practice

1. Show a list of 5 students; display 'Pass' or 'Fail' using `@if` based on marks.
2. Use `@switch` to show a different message for each day of the week.
3. Add a button to the tea menu that removes an item and check that `@empty` appears at the end.

07. Directives, Class and Style Binding

DEFINITION

A **directive** is an instruction that changes how an element looks or behaves. There are three kinds: components (directives with a template), **structural** directives (change the DOM structure, like `@if`) and **attribute** directives (change appearance, like `NgClass`).

The easiest ways to style dynamically are **class binding** `[class.active]="cond"` and **style binding** `[style.color]="value"`. For many classes at once use `NgClass` / `NgStyle` with an object.

SYNTAX

```
<p [class.active]="isActive">...</p>
<p [style.color]="color" [style.font-size.px]="size">...</p>
<p [ngClass]="{ big: isBig, small: !isBig }">...</p>
<p [ngStyle]="{ 'background-color': bg }">...</p>
```

EXAMPLE

```
import { Component, computed, signal } from '@angular/core';
import { NgClass, NgStyle } from '@angular/common';

@Component({
  selector: 'app-root',
  imports: [NgClass, NgStyle],
  template: `
    <!-- class binding: add the "hot" class only when isHot() is true -->
    <p [class.hot]="isHot()">Tea is {{ isHot() ? 'hot' : 'cold' }}</p>

    <!-- NgClass with an object: several classes at once -->
    <p [ngClass]="{ big: size() > 16, small: size() <= 16 }">Size {{ size() }}px</p>

    <!-- style binding with a unit -->
    <p [style.font-size.px]="size()">This text resizes</p>

    <!-- NgStyle with an object -->
    <p [ngStyle]="{ color: textColor(), 'font-weight': 'bold' }">Colour: {{ textColor() }}</p>

    <button id="temp" (click)="isHot.set(!isHot())">Toggle temperature</button>
    <button id="size" (click)="size.update(s => s + 4)">Bigger text</button>
  `,
  styles: `
    .hot { color: crimson; }
    .big { text-decoration: underline; }
  `,
})
export class DynamicStyleComponent {
  isHot = signal(true);
  size = signal(14);
  // computed: derived value that updates automatically
  textColor = computed(() => (this.isHot() ? 'crimson' : 'royalblue'));
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

During Pongal the same doorstep gets a bigger, colourful kolam. The door is the same; only the look changes with the occasion. Class and style bindings change the look with the state.

தமிழ்

பொங்கல் அன்று வீட்டு வாசலில் கோலம் போடுகிறோம். சாதாரண நாட்களில் சிறிய கோலம்; பண்டிகை நாளில் வண்ணக் கோலம். வாசல் அதே வாசல்தான், ஆனால் நாளுக்கு ஏற்ப அதன் தோற்றம் மாறுகிறது.

அதுபோல HTML element அதேதான்; `[class.hot]` அல்லது `[style.color]` மூலம் நிலைக்கு (state) ஏற்ப அதன் நிறம், அளவு, வடிவம் தானாக மாறும். நிலை மாறினால் தோற்றமும் உடனே மாறும் - நாம் கையால் DOM-ஐ மாற்ற வேண்டியதில்லை.

PROS

- Dynamic UI without touching the DOM
- Simple `[class.x]` syntax
- Works with signals

CONS

- Inline styles are hard to maintain
- NgClass/NgStyle need imports

ADVANCED / NEW FEATURES

- Prefer `[class]` and `[style]` bindings over NgClass/NgStyle in new code
- Directive composition API (`hostDirectives`)

Common mistakes

- Using `[class]="'a b'"` and accidentally removing other classes; prefer `[class.name]` for single classes.
- Forgetting units, e.g. `[style.font-size]="16"` instead of `[style.font-size.px]="16"`.
- Forgetting to import `NgClass` / `NgStyle` in a standalone component.

Interview questions

Q1. What are structural vs attribute directives?
Structural directives add/remove elements (`@if`, `@for`); attribute directives change an element's look or behaviour (`NgClass`).

Q2. `[ngClass]` vs `[class.x]`?
`[class.x]` toggles one class with a boolean; `[ngClass]` handles many classes via object, array or string.

Q3. Is `*ngIf` still valid?
Yes but deprecated in favour of `@if`; new code should use the control flow blocks.

Practice

1. Make a dark/light theme toggle that changes the background and text colour of a card.
2. Highlight the row of a list item when the mouse clicks on it (use `[class.selected]`).
3. Create a progress bar whose width is bound to a number from an input slider.

08. Signals (signal, computed, effect)

DEFINITION

A **signal** is a box that holds a value and tells Angular whenever the value changes. You **read** it by calling it like a function (`count()`) and **write** with `.set()` or `.update()`. Because Angular knows exactly which templates read which signals, only the parts that need to change are refreshed.

`computed()` creates a read-only signal **derived** from other signals (it recalculates automatically and caches the result). `effect()` runs code (a side effect, like logging or saving) whenever the signals it reads change.

SYNTAX

```
const count = signal(0);          // create
count();                          // read -> 0
count.set(5);                     // replace
count.update(c => c + 1);         // change based on old value
const double = computed(() => count() * 2);
effect(() => console.log(count()));
```

EXAMPLE

```
import { Component, computed, effect, signal, untracked } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <h3>Idli Order</h3>
    <button id="dec" (click)="dec()">-</button>
    <strong>{{ plates() }}</strong>
    <button id="inc" (click)="inc()">+</button>

    <p>Price per plate: ₹{{ price }}</p>
    <p>Total: ₹{{ total() }}</p>

    <h4>Change history (written by an effect)</h4>
    <ul>
      @for (h of history(); track $index) { <li>{{ h }}</li> }
    </ul>
  `,
})
export class CartSignalsComponent {
  price = 30;
  plates = signal(1); // writable signal

  // computed: derived, read-only, cached
  total = computed(() => this.plates() * this.price);

  history = signal<string[]>([]);

  constructor() {
    // effect: runs once now and again whenever plates() changes
    effect(() => {
      const message = `plates = ${this.plates()}`;
      // untracked: do not re-run this effect when history changes
      untracked(() => this.history.update((h) => [...h, message]));
    });
  }

  inc() { this.plates.update((n) => n + 1); }
  dec() { this.plates.update((n) => Math.max(1, n - 1)); }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

At an idli shop, when you change the number of plates the bill total updates by itself, because the formula 'plates x price' is already known. A signal is the number of plates and `computed` is the bill.

தமிழ்

இட்லிக் கடையில் நீங்கள் ஆர்டர் செய்யும் தட்டுகளின் எண்ணிக்கையை (plates) மாற்றினால், பில் தொகை (total) தானாகவே மாறுகிறது. கடைக்காரர் ஒவ்வொரு முறையும் புதிதாகக் கணக்குப் போட வேண்டியதில்லை - ஏனெனில் 'தட்டு × விலை' என்ற சூத்திரம் அவர் மனதில் இருக்கிறது.

`signal` = தட்டுகளின் எண்ணிக்கை. `computed` = பில் தொகை (தானாக மாறும்). `effect` = ஒவ்வொரு ஆர்டர் மாற்றத்தையும் நோட்டுப் புத்தகத்தில் எழுதி வைப்பது போன்ற கூடுதல் வேலை.

PROS

- Fine-grained updates (fast)
- Simple mental model
- No subscriptions to manage
- Works without Zone.js

CONS

- Different from RxJS, so two models to learn
- Effects can be overused

ADVANCED / NEW FEATURES

- Signals stable (v17-v20)
- `linkedSignal` and `resource` (v19)
- Signal-based inputs, outputs and queries
- Experimental Signal Forms (v21)

Common mistakes

- Forgetting the parentheses when reading a signal (`count` vs `count()`).
- Mutating an array/object inside a signal (`list().push(x)`); create a new one with `update` instead.
- Setting signals inside `computed()`; computed must be pure.

Interview questions

Q1. What is a signal?
A reactive value container; reading it in a template or computed registers a dependency so Angular updates only what changed.

Q2. `computed` vs `effect`?
`computed` derives a value and returns a signal; `effect` performs a side effect and returns nothing.

Q3. `set` vs `update`?
`set` assigns a new value; `update` receives the old value and returns the new one.

Practice

1. Create a `quantity` and `price` signal and a `computed` total including 18% GST.
2. Use `effect` to save a counter value into `localStorage` every time it changes.
3. Build a to-do counter showing the number of completed tasks with `computed`.

09. Component Communication (input, output, model)

DEFINITION

Components rarely work alone. A **parent** passes data to a **child** using an **input** and the child tells the parent something happened using an **output** event. The rule of thumb is: *data goes down, events go up*.

Modern Angular uses the functions `input()`, `input.required()` and `output()`. `model()` creates a writable input that also emits changes, so the parent can use two-way binding `[(name)]`.

SYNTAX

```
// child
title  = input.required<string>();   // value from parent
liked  = output<string>();           // event to parent
count  = model(0);                   // two-way value

// parent template
<app-child [title]="x" (liked)="onLiked($event)" [(count)]="total" />
```

EXAMPLE

```
import { Component, input, model, output, signal } from '@angular/core';

// ---- Child 1: input() and output() ----
@Component({
  selector: 'app-like-button',
  template: `<button (click)="liked.emit(title())">Like "{{ title() }}"</button>`,
})
export class LikeButtonComponent {
  title = input.required<string>(); // data coming IN from the parent
  liked = output<string>();         // events going OUT to the parent
}

// ---- Child 2: model() = two-way binding ----
@Component({
  selector: 'app-stepper',
  template: `
    <button class="minus" (click)="count.set(count() - 1)">-</button>
    {{ count() }}
    <button class="plus" (click)="count.set(count() + 1)">+</button>
  `,
})
export class StepperComponent {
  count = model(0);
}

// ---- Parent ----
@Component({
  selector: 'app-root',
  imports: [LikeButtonComponent, StepperComponent],
  template: `
    <app-like-button title="Angular Basics" (liked)="onLiked($event)" />
    <p>Last liked: {{ last() || 'nothing yet' }}</p>

    <app-stepper [(count)]="total" />
    <p>Parent sees total = {{ total() }}</p>
  `,
})
export class AppComponent {
  last = signal('');
  total = signal(0);

  onLiked(title: string) {
    this.last.set(title);
```

```
  }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A mother (parent) asks a child to bring half a kilo of rice (input); the child comes back and shouts 'got it!' (output). Data flows down, events flow up.

தமிழ்

ஒரு வீட்டில் அம்மா (parent) சமையலறையில் இருக்கும் குழந்தைக்கு (child) 'அரை கிலோ அரிசி எடுத்து வா' என்று சொல்கிறார் - இது **input** (மேலிருந்து கீழே தகவல்). குழந்தை அரிசியுடன் வந்து 'எடுத்துவிட்டேன்!' என்று குரல் கொடுக்கிறது - இது **output** (கீழிருந்து மேலே நிகழ்வு).

இருவரும் ஒரே எண்ணை (எ.கா. மீதமுள்ள அரிசி அளவு) பகிர்ந்து புதுப்பித்துக்கொண்டால் அது **model()** - இருவழி இணைப்பு.

PROS

- Clear one-way data flow
- Type-safe inputs/outputs
- Required inputs checked at compile time

CONS

- Deep trees cause prop drilling (use services)
- Overusing two-way binding hides data flow

ADVANCED / NEW FEATURES

- Function-based `input()`, `output()`, `model()` (stable v19)
- Old `@Input/@Output` still supported

Common mistakes

- Trying to change an input's value inside the child; inputs are read-only (use `model()` if the child must write).
- Forgetting `()` when reading a signal input: `title` vs `title()`.
- Using `$event` wrongly; for `output<string>()`, `$event` is the emitted string.

Interview questions

Q1. How does a parent pass data to a child?
With property binding to an `input()` of the child, e.g. `<app-child [title]="t" />`.

Q2. How does a child notify its parent?
By emitting an `output()` event that the parent handles with `(eventName)="handler($event)"`.

Q3. What is `input.required`?
An input that the parent must provide; the compiler errors if it is missing.

Practice

1. Create a `rating-star` child that takes a `value` input and emits a `rated` event when a star is clicked.
2. Make a `todo-item` child that receives a task object and emits `delete` with the task id.
3. Use `model()` to build a `toggle-switch` that parents can bind with `[(on)]`.

10. Pipes

DEFINITION

A **pipe** transforms a value for display, without changing the original data. You write it in the template with the `|` symbol: `{{ name | uppercase }}`. Pipes can take arguments after a colon and can be chained.

Angular has many built-in pipes: `date`, `currency`, `percent`, `uppercase`, `lowercase`, `slice`, `json`, `async` and more. You can create a **custom pipe** with the `@Pipe` decorator and a `transform` method.

SYNTAX

```
{{ value | pipeName }}
{{ value | pipeName:arg1:arg2 }}
{{ today | date:'dd MMM yyyy' | uppercase }}   // chained

@Pipe({ name: 'myPipe' })
export class MyPipe implements PipeTransform {
  transform(value: string, arg = 1): string { return ...; }
}
```

EXAMPLE

```
import { CurrencyPipe, DatePipe, JsonPipe, PercentPipe, SlicePipe, UpperCasePipe } from
'@angular/common';
import { Component, Pipe, PipeTransform } from '@angular/core';

// A custom pipe: shortens long text
@Pipe({ name: 'truncate' })
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit = 10): string {
    return value.length > limit ? value.slice(0, limit) + '...' : value;
  }
}

@Component({
  selector: 'app-root',
  imports: [CurrencyPipe, DatePipe, JsonPipe, PercentPipe, SlicePipe, UpperCasePipe,
TruncatePipe],
  template: `
    <p>Upper: {{ course | uppercase }}</p>
    <p>Price: {{ price | currency: 'INR' : 'symbol' : '1.2-2' }}</p>
    <p>Date: {{ launch | date: 'dd MMM yyyy' }}</p>
    <p>Discount: {{ discount | percent }}</p>
    <p>First two: {{ cities | slice: 0 : 2 }}</p>
    <p>Truncated: {{ about | truncate: 14 }}</p>
    <p>Chained: {{ launch | date: 'MMMM' | uppercase }}</p>
    <pre>{{ student | json }}</pre>
  `,
})
export class PipesDemoComponent {
  course = 'angular pipes';
  price = 1299.5;
  launch = new Date(2026, 0, 26); // 26 January 2026
```

```
  discount = 0.256;
  cities = ['Chennai', 'Madurai', 'Trichy', 'Salem'];
  about = 'Pipes transform values for display';
  student = { name: 'Arun', marks: [82, 91] };
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A vegetable shop keeps tomato price as 40 in the ledger but prints '₹40.00 per kg' on the bill. The data is unchanged; only the display format changes. That is a pipe.

தமிழ்

காய்கறிக் கடையில் தக்காளி கிலோ ₹40 என்று கடைக்காரர் மனதில் இருக்கிறது. ஆனால் பில்லில் 'தக்காளி - 2 கிலோ - ₹80.00' என்று அழகாக வடிவமைத்து எழுதுகிறார். தக்காளியின் உண்மையான விலை மாறவில்லை; காட்டும் விதம் மட்டுமே மாறியது.

Pipe அதே வேலையைச் செய்கிறது. தரவு (data) அப்படியே இருக்கும்; திரையில் காட்டும்போது தேதி, பணம், சதவீதம் என்று அழகாக மாற்றிக் காட்டும்.

PROS

- Reusable formatting
- Pure pipes are cached
- Keeps components clean

CONS

- Impure pipes can slow the app
- Locale data must be configured for other regions

ADVANCED / NEW FEATURES

- All built-in pipes are standalone
- `toSignal` often replaces the `async` pipe
- `@let` can store pipe results

Common mistakes

- Forgetting to import the pipe (e.g. `CurrencyPipe`) in a standalone component's `imports`.
- Using a pipe to change the original data; pipes only change what is displayed.
- Making a pipe impure or heavy; calling slow code inside `transform` runs often.

Interview questions

Q1. What is a pipe?
A template feature that transforms a value for display, using `|`.

Q2. Pure vs impure pipes?
Pure pipes run only when their input reference changes (default, fast); impure pipes run on every change detection.

Q3. How do you create a custom pipe?
Use `@Pipe({name})` on a class implementing `PipeTransform` with a `transform` method, and import it where needed.

Practice

1. Display a list of prices in INR using `currency` and sort them in the component.
2. Write a custom `initials` pipe: 'Arun Kumar' -> 'AK'.
3. Show today's date in three different formats using the `date` pipe.

Intermediate Level

11. Services and Dependency Injection

DEFINITION

A **service** is a plain class that holds logic or data you want to share (loading data, a shopping cart, logging). Components should focus on the screen; services do the real work.

Dependency Injection (DI) means Angular creates the service for you and hands it over when a class asks for it with `inject(ServiceName)`. With `providedIn: 'root'` there is exactly **one shared instance** for the whole app, so every component sees the same data.

SYNTAX

```
@Injectable({ providedIn: 'root' })
export class DataService {
  getItems() { return ['a', 'b']; }
}

@Component({...})
export class MyComponent {
  private data = inject(DataService);   // ask Angular for it
}
```

EXAMPLE

```
import { Component, computed, inject, Injectable, signal } from '@angular/core';

// A service: ONE shared instance for the whole app (providedIn: 'root')
@Injectable({ providedIn: 'root' })
export class CartService {
  private readonly items = signal<string[]>([]);

  readonly list = this.items.asReadonly(); // others can read, but not change directly
  readonly count = computed(() => this.items().length);

  add(item: string) {
    this.items.update((current) => [...current, item]);
  }

  clear() {
    this.items.set([]);
  }
}

// Component 1 writes to the cart
@Component({
  selector: 'app-product-list',
  template: `
    @for (p of products; track p) {
      <button (click)="cart.add(p)">Add {{ p }}</button>
    }
  `,
})
export class ProductListComponent {
  products = ['Idli Batter', 'Filter Coffee Powder'];
  cart = inject(CartService); // dependency injection
}

// Component 2 reads the SAME cart
@Component({
  selector: 'app-cart-badge',
```

```
  template: `
    <p>Cart items: {{ cart.count() }}</p>
    <ul>
      @for (item of cart.list(); track $index) { <li>{{ item }}</li> }
    </ul>
  `,
})
export class CartBadgeComponent {
  cart = inject(CartService);
}

@Component({
  selector: 'app-root',
  imports: [ProductListComponent, CartBadgeComponent],
  template: `
    <h2>Online Grocery</h2>
    <app-product-list />
    <app-cart-badge />
  `,
})
export class AppComponent {}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A whole village shares one public water tap. A service is that single shared tap; dependency injection is the municipality bringing the connection to your home.

தமிழ்

ஒரு ஊரில் எல்லாரும் ஒரே பொதுக் குடிநீர்க் குழாயைப் பயன்படுத்துகிறார்கள். ஒவ்வொரு வீடும் தனியாக ஆழ்குழாய்க் கிணறு தோண்டத் தேவையில்லை. ஒரு வீட்டில் தண்ணீர் எடுத்தால் மற்ற வீடுகளுக்கும் அதே நீர்த்தொட்டியின் நிலை தெரியும்.

Service அந்தப் பொதுக் குழாய் போல - ஒரே ஒரு பிரதி, எல்லா component-களும் பகிர்ந்துகொள்ளும். **Dependency Injection** என்பது குழாய் இணைப்பை உங்கள் வீடு வரை கொண்டுவந்து கொடுக்கும் நகராட்சி ஊழியர் போல; நாமே கிணறு தோண்ட வேண்டியதில்லை, `inject()` என்று கேட்டால் போதும்.

PROS

- Shared logic and state
- Easy to mock in tests
- Loose coupling

CONS

- Singletons can hide global state
- Wrong provider scope causes confusing bugs

ADVANCED / NEW FEATURES

- `inject()` function (replaces constructor injection)
- `provideAppInitializer`
- Tree-shakable `providedIn: 'root'`

Common mistakes

- Creating a service with `new CartService()` instead of injecting it; you lose the shared instance.
- Forgetting `@Injectable({ providedIn: 'root' })`, which gives a "No provider" error.
- Putting HTTP calls and business logic inside components instead of services.

Interview questions

Q1. What is a service in Angular?
A class that encapsulates reusable logic or state, injected into components and other services.

Q2. What is dependency injection?
A design pattern where a class receives its dependencies from an injector instead of creating them itself.

Q3. What does `providedIn: 'root'` do?
Registers the service in the root injector, creating a single app-wide instance (tree-shakable).

Practice

1. Create a `ThemeService` that stores 'light' or 'dark' and use it in two components.
2. Make a `LoggerService` with `log(message)` and call it from three components.
3. Add a `remove(item)` method to `CartService` and a remove button next to each item.

12. Template-Driven Forms

DEFINITION

In a **template-driven form** you describe the form mostly in HTML. Add `FormsModule`, put `ngModel` and a `name` on each input, and Angular builds the form model behind the scenes.

Add validation with HTML attributes like `required`, `minlength` and `email`. Template reference variables such as `#f="ngForm"` and `#nameCtrl="ngModel"` give you the state: `valid`, `invalid`, `dirty`, `touched` and `errors`. This style is good for small, simple forms.

SYNTAX

```
<form #f="ngForm" (ngSubmit)="save(f.value)">
  <input name="email" [(ngModel)]="model.email" required email #e="ngModel" />
  @if (e.invalid && e.touched) { <small>Invalid email</small> }
  <button [disabled]="f.invalid">Save</button>
</form>
```

EXAMPLE

```
import { Component } from '@angular/core';
import { FormsModule, NgForm } from '@angular/forms';

@Component({
  selector: 'app-root',
  imports: [FormsModule],
  template: `
    <h3>Course Registration</h3>
    <form #f="ngForm" (ngSubmit)="submit(f)">
      <label>Name
        <input id="name" name="name" [(ngModel)]="model.name" required minlength="3"
#nameCtrl="ngModel" />
      </label>
      @if (nameCtrl.invalid && (nameCtrl.dirty || nameCtrl.touched)) {
```

```
        <small>
          @if (nameCtrl.errors?.['required']) { Name is required. }
          @if (nameCtrl.errors?.['minlength']) { Use at least 3 letters. }
        </small>
      }

      <label>Email
        <input id="email" name="email" type="email" [(ngModel)]="model.email" required email
/>
      </label>

      <button type="submit" [disabled]="f.invalid">Register</button>
    </form>

    <p>Form valid: {{ f.valid }}</p>
    @if (message) { <p>{{ message }}</p> }
  `,
})
export class RegisterFormComponent {
  model = { name: '', email: '' };
  message = '';

  submit(form: NgForm) {
    this.message = `Registered ${form.value.name} (${form.value.email})`;
    form.resetForm();
  }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A railway reservation form is printed with boxes for name and age. If you hand it over with the name box empty, the counter rejects it. Required/minlength attributes act like that counter clerk.

தமிழ்

ரயில்வே முன்பதிவுப் படிவத்தை எடுத்துக்கொள்ளுங்கள். பெயர், வயது, பயண நாள் என்று ஒவ்வொரு கட்டமும் அச்சிடப்பட்ட காகிதம் அங்கேயே இருக்கிறது. பெயர் கட்டத்தை நிரப்பாமல் கவுண்டரில் கொடுத்தால், 'பெயர் எழுதுங்கள்' என்று உடனே திருப்பிக் கொடுத்துவிடுவார்கள்.

Template-driven form-இல் படிவத்தின் அமைப்பு HTML-இலேயே இருக்கும் (`required` , `minlength`). எல்லாக் கட்டங்களும் சரியாக இருந்தால் மட்டுமே **Register** பொத்தான் வேலை செய்யும் - கவுண்டர் ஊழியர் சரிபார்த்த பிறகே ஏற்றுக்கொள்வது போல.

PROS

- Fast to write for simple forms
- Little TypeScript code
- Familiar HTML validation attributes

CONS

- Hard to test
- Not suitable for dynamic or large forms

- Logic spread across the template

ADVANCED / NEW FEATURES

- Signal Forms (experimental in v21) aim to simplify form handling
- Unified control state events (v18)

Common mistakes

- Forgetting the `name` attribute on an `ngModel` input; the control will not register in the form.
- Not importing `FormsModule`.
- Showing errors immediately on a pristine form; check `dirty` or `touched` first.

Interview questions

Q1. What are template-driven forms?
Forms built mainly in the template with `ngModel`, where Angular creates the form model automatically.

Q2. Why is `name` needed on inputs inside a form?
NgForm registers each `ngModel` control by its `name` attribute.

Q3. What do `dirty`, `touched`, `valid` mean?
`dirty`: value changed; `touched`: input was focused then left; `valid`: all validators pass.

Practice

1. Build a feedback form with name, rating (select) and comments (required, minimum 10 characters).
2. Disable the submit button until a 'I agree' checkbox is ticked.
3. Show a red border on invalid inputs using `[class.invalid]`.

13. Reactive Forms

DEFINITION

In **reactive forms** you create the form model in the **component class** using `FormGroup` and `FormControl` (or the shorter `FormBuilder`). The template simply connects to it with `[formGroup]` and `formControlName`. Everything is explicit, typed and easy to test.

Use `fb.nonNullable.group({...})` to get strongly typed values. Validators are plain functions such as `Validators.required` and `Validators.email`. You can read the whole value with `form.value` or `form.getRawValue()` and react to changes with `valueChanges`.

SYNTAX

```
form = inject(FormBuilder).nonNullable.group({
  name:  ['', [Validators.required, Validators.minLength(3)]],
  email: ['', [Validators.required, Validators.email]],
});

<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="name" />
</form>
```

EXAMPLE

```
import { Component, inject, signal } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';

@Component({
  selector: 'app-root',
  imports: [ReactiveFormsModule],
  template: `
    <h3>Student Profile</h3>
    <form [formGroup]="form" (ngSubmit)="submit()">
      <input id="name" formControlName="name" placeholder="Name" />
      @if (form.controls.name.touched && form.controls.name.invalid) { <small>Name needs 3+
letters.</small> }

      <input id="email" formControlName="email" placeholder="Email" />
      @if (form.controls.email.dirty && form.controls.email.invalid) { <small>Enter a valid
email.</small> }

      <select formControlName="city">
        <option>Chennai</option>
        <option>Madurai</option>
        <option>Coimbatore</option>
      </select>

      <button type="submit" [disabled]="form.invalid">Save</button>
    </form>

    <p>Form status: {{ form.status }}</p>
    @if (saved()) { <pre>{{ saved() }}</pre> }
  `,
})
export class ReactiveFormComponent {
  private fb = inject(FormBuilder);

  // The form model lives in the class and is fully typed
  form = this.fb.nonNullable.group({
    name: ['', [Validators.required, Validators.minLength(3)]],
    email: ['', [Validators.required, Validators.email]],
    city: ['Chennai'],
  });

  saved = signal('');

  submit() {
    if (this.form.valid) {
      this.saved.set(JSON.stringify(this.form.getRawValue()));
    }
  }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Before printing wedding invitations you plan every field and rule with the designer first; the printer just follows. Reactive forms define the rules in code first, the template follows.

தமிழ்

திருமண அழைப்பிதழை அச்சடிக்கும் முன், மணமக்கள் பெயர், தேதி, மண்டபம் என்று எல்லா அமைப்பையும் முதலிலேயே வடிவமைப்பாளரிடம் திட்டமிட்டுக் கொடுப்போம்; அச்சகம் அதை அப்படியே பின்பற்றும். அச்சடித்த பின் காகிதத்தில் குழப்பம் வராது.

Reactive form-உம் அப்படியே: படிவத்தின் முழு விதிகளையும் (எந்தக் கட்டம், என்ன சரிபார்ப்பு) TypeScript-இல் முன்கூட்டியே எழுதிவிடுகிறோம்; HTML அதைப் பின்பற்றுகிறது. பெரிய படிவங்களுக்கும் சோதனை செய்வதற்கும் இது எளிது.

PROS

- Explicit and testable
- Strongly typed
- Great for dynamic forms (FormArray)
- Observable `valueChanges`

CONS

- More code than template-driven
- Steeper learning curve

ADVANCED / NEW FEATURES

- Typed forms (v14)
- `FormRecord`
- `events` stream on controls (v18)
- Signal Forms experimental

Common mistakes

- Forgetting to import `ReactiveFormsModule` (error: Can't bind to 'formGroup').
- Using `formControlName` outside an element that has `[formGroup]`.
- Reading `form.value` when you need disabled controls too; use `getRawValue()`.

Interview questions

Q1. What are reactive forms?
Forms whose model (FormGroup/FormControl) is created in the component class and bound to the template.

Q2. What is FormBuilder?
A helper service with shorthand methods (`group`, `control`, `array`) to build form models.

Q3. What is `valueChanges`?
An Observable that emits whenever a control or group value changes.

Practice

1. Add a `phone` control that accepts exactly 10 digits using `Validators.pattern`.
2. Show a live preview card using `form.valueChanges` (hint: `toSignal`).
3. Add a `Reset` button that restores the default values with `form.reset()`.

14. Form Validation and Custom Validators

DEFINITION

A **validator** is a function that checks a control and returns `null` if the value is fine, or an **error object** (like `{ spaces: true }`) if it is not. Built-in validators cover common cases; for everything

else you write your own.

A **cross-field validator** is attached to the whole `FormGroup` so it can compare several fields, for example *password* and *confirm password*. Errors then live on `form.errors` instead of on a single control.

SYNTAX

```
const noSpaces: ValidatorFn = (control) =>
  /\s/.test(control.value) ? { spaces: true } : null;

new FormGroup({ ... }, { validators: passwordsMatch });   // group-level

form.controls.username.errors?.['spaces']
form.errors?.['mismatch']
```

EXAMPLE

```
import { Component } from '@angular/core';
import { AbstractControl, FormControl, FormGroup, ReactiveFormsModule, ValidationErrors,
ValidatorFn, Validators } from '@angular/forms';

// 1. Custom validator for ONE control: returns null (valid) or an error object
const noSpaces: ValidatorFn = (control: AbstractControl): ValidationErrors | null =>
  /\s/.test(control.value) ? { spaces: true } : null;

// 2. Cross-field validator for the WHOLE group
const passwordsMatch: ValidatorFn = (group: AbstractControl): ValidationErrors | null => {
  const password = group.get('password')?.value;
  const confirm = group.get('confirm')?.value;
  return password === confirm ? null : { mismatch: true };
};

@Component({
  selector: 'app-root',
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form">
      <input id="username" formControlName="username" placeholder="Username" />
      @if (form.controls.username.errors?.['spaces']) { <small>No spaces allowed.</small> }

      <input id="password" type="password" formControlName="password" placeholder="Password"
/>
      @if (form.controls.password.errors?.['minlength']) { <small>Minimum 6 characters.
</small> }

      <input id="confirm" type="password" formControlName="confirm" placeholder="Confirm" />
      @if (form.errors?.['mismatch'] && form.controls.confirm.dirty) { <small>Passwords do not
match.</small> }
    </form>
    <p>{{ form.valid ? 'Form is valid' : 'Form is invalid' }}</p>
  `,
})
export class SignupValidatorsComponent {
  form = new FormGroup(
    {
      username: new FormControl('', { nonNullable: true, validators: [Validators.required,
noSpaces] }),
      password: new FormControl('', { nonNullable: true, validators: [Validators.required,
Validators.minLength(6)] }),
      confirm: new FormControl('', { nonNullable: true }),
    },
    { validators: passwordsMatch },
  );
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

When opening a bank account you type the password twice. A rule that compares both fields tells you 'passwords do not match'. That is a cross-field validator.

தமிழ்

வங்கியில் புதிய கணக்குத் தொடங்கும்போது, கடவுச்சொல்லை இரண்டு முறை எழுதச் சொல்வார்கள். இரண்டும் ஒன்றாக இல்லையென்றால் 'பொருந்தவில்லை' என்று சொல்லி மீண்டும் எழுதச் சொல்வார்கள். ஒரே கட்டத்தை மட்டும் பார்த்து இதை முடிவு செய்ய முடியாது - இரண்டு கட்டங்களையும் ஒப்பிட வேண்டும்.

தனிக் கட்டத்துக்கான விதி (இடைவெளி கூடாது) = custom validator. இரண்டு கட்டங்களை ஒப்பிடும் விதி = cross-field validator; அது முழு `FormGroup`-இல் இணைக்கப்படும்.

PROS

- Reusable rules
- Cross-field checks
- Async validators for server checks

CONS

- Group errors are easy to forget to display
- Complex validators need good tests

ADVANCED / NEW FEATURES

- Typed validators with typed forms
- Schema based validation in experimental Signal Forms

Common mistakes

- Returning `false` or `true` from a validator; it must return `null` or an error object.
- Putting a cross-field validator on one control, where it cannot see the other control.
- Showing all errors at once; show the first relevant message per field.

Interview questions

Q1. What does a validator function return?
`null` when valid, otherwise an object describing the error, e.g. `{ spaces: true }`.

Q2. How do you validate two fields together?
Attach a validator to the parent `FormGroup` that reads both controls.

Q3. What is an async validator?
A validator returning an Observable/Promise (e.g. check username availability on the server).

Practice

1. Write a `strongPassword` validator that needs a digit and an uppercase letter.

2. Add a cross-field validator ensuring `endDate` is after `startDate`.
3. Create a validator factory `forbiddenWord(word)` that rejects a given word in a comment box.

15. Routing Basics

DEFINITION

The **Angular Router** maps URLs to components, so a single-page app can still have many 'pages'. You define an array of **routes** (`path` + `component`), register it with `provideRouter(routes)`, place a `<router-outlet />` where the page should appear, and use `routerLink` for navigation without a reload.

A route with `path: '**'` matches everything else and is perfect for a *404 Not Found* page. `routerLinkActive` adds a CSS class to the link of the page you are currently on.

SYNTAX

```
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent, title: 'About' },
  { path: '**', component: NotFoundComponent },
];

provideRouter(routes)           // in app.config.ts

<a routerLink="/about" routerLinkActive="active">About</a>
<router-outlet />
```

EXAMPLE

```
import { ApplicationConfig, Component } from '@angular/core';
import { provideRouter, RouterLink, RouterLinkActive, RouterOutlet, Routes, withHashLocation }
from '@angular/router';

// Three simple "pages"
@Component({ selector: 'app-home', template: `<h2>Home</h2><p>Welcome to Koyambedu Bus Stand.
</p>` })
export class HomeComponent {}

@Component({ selector: 'app-about', template: `<h2>About</h2><p>We run buses across Tamil
Nadu.</p>` })
export class AboutComponent {}

@Component({ selector: 'app-not-found', template: `<h2>404 - Page not found</h2>` })
export class NotFoundComponent {}

// Route table: path -> component. '**' (wildcard) must be LAST.
const routes: Routes = [
  { path: '', component: HomeComponent, title: 'Home' },
  { path: 'about', component: AboutComponent, title: 'About' },
  { path: '**', component: NotFoundComponent },
];

// In a real app this lives in app.config.ts (withHashLocation is only to keep URLs simple
here)
export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes, withHashLocation())],
};

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav>
```

```
      <a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{ exact: true
}">Home</a> |
      <a routerLink="/about" routerLinkActive="active">About</a> |
      <a routerLink="/nowhere">Broken link</a>
    </nav>
    <router-outlet />
  `,
  styles: `.active { font-weight: bold; }`,
})
export class AppComponent {}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Koyambedu bus stand is one station, but each platform number leads to a different bus. A URL is the platform number; the matching component is the bus.

தமிழ்

கோயம்பேடு பேருந்து நிலையத்தை நினைத்துப் பாருங்கள். நிலையம் ஒன்றுதான்; ஆனால் 'மதுரை - நடைமேடை 5', 'கோவை - நடைமேடை 8' என்று ஒவ்வொரு நடைமேடைக்கும் ஒரு எண் உண்டு. அந்த எண்ணுக்குப் போனால் அந்த ஊருக்கான பேருந்து நிற்கும்.

Router அந்த அறிவிப்புப் பலகை போல: URL (நடைமேடை எண்) → component (பேருந்து). இல்லாத எண்ணுக்குப் போனால் 'அப்படி ஒரு நடைமேடை இல்லை' என்று காட்டும் - அதுதான் `**` route (404).

PROS

- Deep links and browser back/forward work
- Lazy loading support
- Guards and resolvers built in

CONS

- Misconfigured routes are hard to debug
- Hash vs path URLs need server setup

ADVANCED / NEW FEATURES

- Standalone `provideRouter`
- View transitions (`withViewTransitions`)
- Functional guards
- `redirectTo` can be a function (v18.1)

Common mistakes

- Forgetting `<router-outlet />`; routes match but nothing appears.
- Using `href="/about"` instead of `routerLink`, which reloads the entire page.
- Placing the `**` wildcard route before other routes (it matches first).

Interview questions

Q1. What is the Angular Router?
A library mapping URLs to components, enabling navigation in a single-page app without reloads.

Q2. `routerLink` vs `href`?
`routerLink` navigates inside the SPA (no reload); `href` makes the browser request the page again.

Q3. What does `path: '**'` do?
It is the wildcard route that matches any URL not matched earlier, used for a 404 page.

Practice

1. Create Home, Products and Contact pages with a navigation bar.
2. Add a redirect: `{ path: 'home', redirectTo: '', pathMatch: 'full' }`.
3. Highlight the active link using `routerLinkActive` and a CSS class.

16. Route Parameters, Query Params and Child Routes

DEFINITION

A **route parameter** is a changing part of the URL, written `:id` in the path (`/courses/2`). **Query parameters** come after `?` (`/courses/2?tab=syllabus`). With `withComponentInputBinding()` the router passes both straight into component `input()`s - no `ActivatedRoute` needed.

Child routes nest pages inside a parent layout: the parent has its own `<router-outlet />` and the `children` array decides what appears inside it.

SYNTAX

```
{ path: 'courses', component: CoursesLayout, children: [
    { path: '',    component: CourseList },
    { path: ':id', component: CourseDetail },
] }

provideRouter(routes, withComponentInputBinding())

id  = input.required<string>();     // from :id
tab = input('overview');            // from ?tab=...

<a [routerLink]="['/courses', 2]" [queryParams]="{ tab: 'syllabus' }">
```

EXAMPLE

```
import { ApplicationConfig, Component, computed, input } from '@angular/core';
import { provideRouter, RouterLink, RouterOutlet, Routes, withComponentInputBinding,
withHashLocation } from '@angular/router';

const COURSES = [
  { id: 1, title: 'Angular Basics' },
  { id: 2, title: 'TypeScript Deep Dive' },
];

// Parent layout: its own <router-outlet> shows the child routes
@Component({
  selector: 'app-courses-layout',
  imports: [RouterOutlet],
  template: `<h2>Courses</h2><router-outlet />`,
})
export class CoursesLayoutComponent {}

@Component({
  selector: 'app-course-list',
  imports: [RouterLink],
```

```
  template: `
    <ul>
      @for (c of courses; track c.id) {
        <li><a [routerLink]="[c.id]" [queryParams]="{ tab: 'syllabus' }">{{ c.title }}</a>
</li>
      }
    </ul>
  `,
})
export class CourseListComponent {
  courses = COURSES;
}

@Component({
  selector: 'app-course-detail',
  imports: [RouterLink],
  template: `
    <h3>Course #{{ id() }}: {{ course()?.title ?? 'Unknown' }}</h3>
    <p>Showing tab: {{ tab() }}</p>
    <a routerLink="/courses">Back to list</a>
  `,
})
export class CourseDetailComponent {
  // Filled from the URL thanks to withComponentInputBinding()
  id = input.required<string>();   // path param  :id
  tab = input('overview');         // query param ?tab=
  course = computed(() => COURSES.find((c) => c.id === Number(this.id())));
}

const routes: Routes = [
  { path: '', redirectTo: 'courses', pathMatch: 'full' },
  {
    path: 'courses',
    component: CoursesLayoutComponent,
    children: [
      { path: '', component: CourseListComponent },
      { path: ':id', component: CourseDetailComponent },
    ],
  },
];

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes, withComponentInputBinding(), withHashLocation())],
};

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  template: `<router-outlet />`,
})
export class AppComponent {}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A postal address 'Anna Nagar / house 12': the area is the layout and 12 is the changing part (parameter). A query parameter is a note like 'urgent'.

தமிழ்

தபால் முகவரியை எடுத்துக்கொள்ளுங்கள்: 'அண்ணா நகர் / 12-ஆம் எண் வீடு'. 'அண்ணா நகர்' என்பது பெற்றோர் (layout) - அந்தப் பகுதியின் பொதுவான அமைப்பு. '12' என்பது மாறக்கூடிய பகுதி (`:id` parameter) - ஒவ்வொரு வீட்டுக்கும் வேறு எண்.

கடைசியில் 'தபால் பெட்டி எண்: 5, அவசரம்' என்று கூடுதல் குறிப்பு எழுதுவது போல `?tab=syllabus` என்பது query parameter. முகவரி மாறினாலும் தபால்காரர் (router) சரியான வீட்டைக் கண்டுபிடிப்பார்.

PROS

- Shareable URLs
- Params bound directly to inputs
- Nested layouts via child routes

CONS

- Params are always strings
- Easy to forget outlet in parent routes

ADVANCED / NEW FEATURES

- `withComponentInputBinding()` binds params, query and data to inputs
- Router state signals (`toSignal`)

Common mistakes

- Forgetting `withComponentInputBinding()` ; inputs stay empty.
- Giving the parent route a component without a `<router-outlet />` ; children never show.
- Reading `id` as a number; route params are always strings (convert with `Number()`).

Interview questions

Q1. Route params vs query params?
Route params are part of the path (`/courses/2` , required for matching); query params follow `?` (optional, e.g. filters).

Q2. How do you read a route parameter?
Via `input()` with `withComponentInputBinding()` , or `ActivatedRoute.paramMap` / `snapshot` .

Q3. What are child routes?
Routes nested under a parent route; the parent's `<router-outlet />` displays them.

Practice

1. Add a `/products/:id` route and show the product's name and price from an array.
2. Add a `?sort=price` query parameter that sorts the course list.
3. Create a `/admin` layout with child routes `users` and `settings` .

17. Route Guards and Lazy Loading

DEFINITION

A **route guard** is a function that decides whether a navigation is allowed. `canActivate` runs before a route is entered; it returns `true` to allow, `false` to block, or a `UrlTree` to **redirect** (for example to a login page). Guards are plain functions that can use `inject()` .

Lazy loading means the code of a page is downloaded only when the user first visits it, using `loadComponent` (or `loadChildren`). This makes the first load smaller and faster.

SYNTAX

```
export const authGuard: CanActivateFn = () => {
  const ok = inject(AuthService).loggedIn();
  return ok ? true : inject(Router).createUrlTree(['/login']);
};

{ path: 'admin',
  canActivate: [authGuard],
  loadComponent: () => import('./admin.component').then(m => m.AdminComponent) }
```

EXAMPLE

```
import { ApplicationConfig, Component, inject, Injectable, signal } from '@angular/core';
import { CanActivateFn, provideRouter, Router, RouterLink, RouterOutlet, Routes,
withHashLocation } from '@angular/router';

@Injectable({ providedIn: 'root' })
export class AuthService {
  loggedIn = signal(false);
}

// Functional guard: true = allow, UrlTree = redirect
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.loggedIn() ? true : router.createUrlTree(['/login']);
};

@Component({ selector: 'app-home', template: `<h2>Home</h2><p>Public page.</p>` })
export class HomeComponent {}

@Component({
  selector: 'app-login',
  template: `
    <h2>Login required</h2>
    <button id="login" (click)="login()">Log in</button>
  `,
})
export class LoginComponent {
  private auth = inject(AuthService);
  private router = inject(Router);

  login() {
    this.auth.loggedIn.set(true);
    this.router.navigate(['/admin']);
  }
}

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  {
    path: 'admin',
    canActivate: [authGuard],
    // Lazy loaded: admin.component.ts becomes a separate JavaScript chunk
    loadComponent: () => import('./admin.component').then((m) => m.AdminComponent),
  },
];

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes, withHashLocation())],
};

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink],
  template: `
    <nav>
```

```
      <a routerLink="/">Home</a> |
      <a routerLink="/admin">Admin</a>
    </nav>
    <router-outlet />
  `,
})
export class AppComponent {}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

At a temple special-darshan entrance the staff checks your ticket (guard). Prasadam for each shrine is cooked only when devotees arrive there (lazy loading).

தமிழ்

கோயிலில் சிறப்பு தரிசனத்துக்குச் செல்லும்போது வாசலில் உள்ள ஊழியர் உங்கள் டிக்கெட்டைப் பார்ப்பார். டிக்கெட் இருந்தால் உள்ளே அனுமதி; இல்லையென்றால் 'டிக்கெட் கவுண்டருக்குப் போங்கள்' என்று திருப்பி அனுப்புவார். அவர்தான் **route guard**; கவுண்டருக்கு அனுப்புவது = `UrlTree` redirect.

ஒவ்வொரு சன்னதிக்கும் உரிய பிரசாதம் அந்தச் சன்னதிக்கு பக்தர் வந்த பிறகே தயாரிக்கப்பட்டால், சமையல்கூடம் வீணாகாது. அதுபோல **lazy loading** - பயனர் அந்தப் பக்கத்துக்குப் போனால் மட்டுமே அதன் code பதிவிறக்கம் ஆகும்.

PROS

- Smaller first load
- Central access control
- Redirects with UrlTree

CONS

- Guards are not real security
- Lazy chunks add a small delay on first visit

ADVANCED / NEW FEATURES

- Functional guards (`CanActivateFn`)
- `canMatch`
- `@defer` for lazy template parts
- Prefetching with `withPreloading`

Common mistakes

- Returning `false` from a guard without redirecting, leaving the user on a blank screen.
- Relying on guards for security. They only hide UI; the server must still check permissions.
- Using eager `component:` for big rarely-used pages instead of `loadComponent`.

Interview questions

Q1. What is a route guard?
A function that controls navigation: `canActivate`, `canDeactivate`, `canMatch`, `resolve` etc.

Q2. How do you redirect from a guard?
Return a `UrlTree` created with `router.createUrlTree([...])`.

Q3. What is lazy loading?
Loading route code on demand through dynamic `import()`, reducing the initial bundle size.

Practice

1. Add a `canDeactivate` guard that asks for confirmation when leaving a form with unsaved changes.
2. Make a `roleGuard` that allows only users whose role is 'admin'.
3. Lazy load a `Reports` page and open the browser Network tab to see the extra chunk download.

18. HttpClient and Working with APIs

DEFINITION

HttpClient is Angular's service for talking to servers over HTTP (REST APIs). Enable it with `provideHttpClient()`, inject it, and call `get`, `post`, `put`, `delete`. Each call returns an **Observable**: nothing is sent until you `subscribe`.

Real apps always handle three states: **loading** (waiting), **success** (data arrived) and **error** (network or server failure). Put API calls in a **service**, and give request/response data a TypeScript interface.

SYNTAX

```
provideHttpClient()                       // app.config.ts

private http = inject(HttpClient);
this.http.get<Todo[]>(url).subscribe({
  next:  (data) => ...,
  error: (err)  => ...,
});
this.http.post<Todo>(url, body)
```

EXAMPLE

```
import { HttpClient, provideHttpClient } from '@angular/common/http';
import { ApplicationConfig, Component, inject, Injectable, signal } from '@angular/core';

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient()], // enables HttpClient
};

interface Todo {
  id: number;
  title: string;
  done: boolean;
}

// All API calls live in a service
@Injectable({ providedIn: 'root' })
export class TodoApi {
  private http = inject(HttpClient);
  private base = 'https://api.example.com'; // replace with your real API

  list() {
    return this.http.get<Todo[]>(`${this.base}/todos`);
```

```
  }
  add(title: string) {
    return this.http.post<Todo>(`${this.base}/todos/add`, { title });
  }
  broken() {
    return this.http.get<Todo[]>(`${this.base}/broken`);
  }
}

@Component({
  selector: 'app-root',
  template: `
    <button id="load" (click)="load()">Load todos</button>
    <button id="add" (click)="add()">Add todo</button>
    <button id="broken" (click)="callBroken()">Call broken URL</button>

    @if (loading()) { <p>Loading...</p> }
    @if (error()) { <p>Error: {{ error() }}</p> }

    <ul>
      @for (t of todos(); track t.id) {
        <li>{{ t.done ? '[x]' : '[ ]' }} {{ t.title }}</li>
      }
    </ul>
  `,
})
export class TodoApiComponent {
  private api = inject(TodoApi);
  todos = signal<Todo[]>([]);
  loading = signal(false);
  error = signal('');

  load() {
    this.loading.set(true);
    this.error.set('');
    this.api.list().subscribe({
      next: (data) => {
        this.todos.set(data);
        this.loading.set(false);
      },
      error: () => {
        this.error.set('Could not load todos');
        this.loading.set(false);
      },
    });
  }

  add() {
    this.api.add('Buy jasmine flowers').subscribe((todo) => this.todos.update((list) =>
[...list, todo]));
  }

  callBroken() {
    this.error.set('');
    this.api.broken().subscribe({ error: () => this.error.set('Server returned an error
(500)') });
  }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

In a restaurant you order masala dosa (request); the kitchen prepares it and sends it (response). While waiting you see 'preparing' (loading); if batter is over you are told (error).

தமிழ்

நீங்கள் ஒரு உணவகத்தில் அமர்ந்து சர்வரிடம் 'ஒரு மசாலா தோசை' என்று ஆர்டர் கொடுக்கிறீர்கள் - இது **request**. சமையலறை (server) தோசை தயாரித்து அனுப்புகிறது - இது **response**. இது உடனே நடக்காது; நேரம் ஆகும் (asynchronous).

காத்திருக்கும்போது 'தயாராகிக்கொண்டிருக்கிறது...' (loading). தோசை தயாராகிவிட்டால் தட்டில் வரும் (success). 'மாவு தீர்ந்துவிட்டது' என்றால் (error) நமக்குத் தெரிவிக்க வேண்டும். `HttpClient` இந்த சர்வர் வேலையைச் செய்கிறது.

PROS

- Typed requests/responses
- Interceptors
- Easy testing with mocks
- Works with RxJS

CONS

- Observables must be subscribed
- Boilerplate for loading/error states

ADVANCED / NEW FEATURES

- `provideHttpClient(withFetch())`
- Functional interceptors
- `httpResource` (experimental) for signal-based requests

Common mistakes

- Not subscribing (or not using `async` pipe / `toSignal`); an HTTP Observable does nothing until subscribed.
- Ignoring errors; always handle the `error` callback and show a message.
- Forgetting `provideHttpClient()` (error: No provider for HttpClient).

Interview questions

Q1. What is HttpClient?
Angular's injectable service for HTTP requests, returning typed Observables.

Q2. Why must you subscribe to an HTTP call?
Observables are lazy; the request is sent only on subscription.

Q3. How do you handle errors?
Use the `error` callback in `subscribe`, or the RxJS `catchError` operator in the pipe.

Practice

1. Add a `delete(id)` method and a delete button for each todo.
2. Show a spinner text while loading using `loading()`.
3. Use `catchError` + `of([])` inside the service so the component never crashes on error.

19. RxJS Fundamentals

DEFINITION

RxJS is a library for working with **streams of values over time**. An **Observable** describes a stream; nothing happens until you **subscribe**. **Operators** (`map`, `filter`, `take`...) are chained inside `pipe()` to transform the stream.

In templates, the `async` pipe subscribes for you and unsubscribes automatically, so you avoid memory leaks. A **Subject** is an Observable you can push values into manually (useful for events).

SYNTAX

```
source$.pipe(
  filter(n => n % 2 === 0),
  map(n => n * 10),
).subscribe(value => console.log(value));

{{ source$ | async }}      // template: auto subscribe/unsubscribe
```

EXAMPLE

```
// Run with: npx tsx rxjs-basics.ts
import { filter, interval, lastValueFrom, map, Observable, of, Subject, take, tap } from
'rxjs';

async function main() {
  // 1. Creation + operators in pipe()
  of(1, 2, 3, 4, 5, 6)
    .pipe(
      filter((n) => n % 2 === 0), // keep even numbers
      map((n) => n * 10), // multiply by 10
    )
    .subscribe((v) => console.log('even x10:', v));

  // 2. Build your own Observable (note the cleanup function)
  const greeting$ = new Observable<string>((subscriber) => {
    subscriber.next('Hello');
    subscriber.next('Angular');
    subscriber.complete();
    return () => console.log('cleanup done');
  });
  greeting$.subscribe({
    next: (v) => console.log('got:', v),
    complete: () => console.log('stream completed'),
  });

  // 3. Subject: push values manually; every subscriber receives them
  const bell$ = new Subject<string>();
  bell$.subscribe((m) => console.log('Student A heard:', m));
  bell$.subscribe((m) => console.log('Student B heard:', m));
  bell$.next('Class over!');

  // 4. Timed stream: 3 ticks, 50 ms apart, then it completes
  const last = await lastValueFrom(
    interval(50).pipe(
      take(3),
      map((i) => i + 1),
      tap((n) => console.log('tick', n)),
    ),
  );
  console.log('last tick was', last);
}

main();
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Sun TV live is always streaming; you see it only when you switch the TV on (subscribe) and it stops when you switch off (unsubscribe). Filtering ads is `filter`, converting each scene to Tamil is `map`.

தமிழ்

சன் டிவி நேரலை ஒளிபரப்பு எப்போதும் ஓடிக்கொண்டே இருக்கிறது. நீங்கள் டிவியை ஆன் செய்தால்தான் (**subscribe**) அதைப் பார்க்க முடியும்; ஆஃப் செய்தால் (**unsubscribe**) உங்களுக்கு வருவது நின்றுவிடும். ஒளிபரப்பு = **Observable**.

விளம்பர இடைவேளையை நீக்கிப் பார்க்க விரும்பினால் அது `filter`; ஒவ்வொரு காட்சியையும் தமிழில் மாற்றிப் பார்த்தால் அது `map`. இவை எல்லாம் ஒளிபரப்பு வரும் வழியிலேயே (`pipe`) நடக்கும்.

PROS

- Powerful operators
- Cancellable async work
- Great for events and streams

CONS

- Steep learning curve
- Easy to leak subscriptions
- Overkill for simple state

ADVANCED / NEW FEATURES

- `toSignal` / `toObservable` interop
- `takeUntilDestroyed`
- `rxResource`

Common mistakes

- Forgetting to `subscribe`; nothing runs.
- Subscribing manually in components and never unsubscribing (memory leaks). Prefer `async` pipe, `toSignal` or `takeUntilDestroyed`.
- Nesting `subscribe` inside `subscribe`; use `switchMap` and friends instead.

Interview questions

Q1. What is an Observable?
A lazy stream of zero or more values over time that a consumer can subscribe to.

Q2. Observable vs Promise?
A Promise gives one value, is eager and cannot be cancelled; an Observable can emit many values, is lazy and cancellable.

Q3. What is a Subject?
An Observable that is also an Observer: you call `next()` on it to multicast values to all subscribers.

Practice

1. Use `interval(1000)` with `take(10)` to build a countdown that prints 10 to 1.
2. From `of(1..20)` keep numbers divisible by 3 and square them.
3. Create a `Subject<string>` chat service and subscribe from two components.

20. Component Lifecycle Hooks

DEFINITION

Every component has a **life**: Angular creates it, updates it and finally destroys it. **Lifecycle hooks** are methods Angular calls at these moments. The most important are: `ngOnChanges` (an input changed), `ngOnInit` (ready - good place for setup), `ngAfterViewInit` (the view exists) and `ngOnDestroy` (cleanup).

Use `constructor` only for injecting dependencies. Do initial work in `ngOnInit`, and release timers, event listeners and subscriptions in `ngOnDestroy` to avoid memory leaks.

SYNTAX

```
export class MyComponent implements OnInit, OnDestroy {
  ngOnInit()    { /* load data, start timers */ }
  ngOnDestroy() { /* unsubscribe, clear timers */ }
}
```

EXAMPLE

```
import { AfterViewInit, Component, inject, Injectable, input, OnChanges, OnDestroy, OnInit,
signal, SimpleChanges } from '@angular/core';

// A tiny service so we can SEE the order of the hooks on screen
@Injectable({ providedIn: 'root' })
export class LogService {
  logs = signal<string[]>([]);
  add(message: string) {
    this.logs.update((l) => [...l, message]);
  }
}

@Component({
  selector: 'app-tea-card',
  template: `<p>Tea: {{ name() }}</p>`,
})
export class TeaCardComponent implements OnChanges, OnInit, AfterViewInit, OnDestroy {
  name = input.required<string>();
  private log = inject(LogService);

  constructor() {
    this.log.add('constructor');
  }
  ngOnChanges(changes: SimpleChanges) {
    this.log.add(`ngOnChanges -> ${changes['name'].currentValue}`);
  }
  ngOnInit() {
    this.log.add('ngOnInit');
  }
  ngAfterViewInit() {
    this.log.add('ngAfterViewInit');
  }
  ngOnDestroy() {
```

```
    this.log.add('ngOnDestroy');
  }
}

@Component({
  selector: 'app-root',
  imports: [TeaCardComponent],
  template: `
    <button id="rename" (click)="tea.set('Ginger Tea')">Rename tea</button>
    <button id="toggle" (click)="show.set(!show())">{{ show() ? 'Remove' : 'Show' }}
card</button>

    @if (show()) { <app-tea-card [name]="tea()" /> }

    <h4>Hook log</h4>
    <ol>
      @for (l of logger.logs(); track $index) { <li>{{ l }}</li> }
    </ol>
  `,
})
export class LifecycleLoggerComponent {
  logger = inject(LogService);
  show = signal(true);
  tea = signal('Masala Tea');
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A tea shop's day: open the shop (`ngOnInit`), update the price board when prices change (`ngOnChanges`), and at night switch off the stove and wash the vessels (`ngOnDestroy`).

தமிழ்

ஒரு தேநீர்க் கடையின் நாளை நினைத்துப் பாருங்கள். காலையில் கடையைத் திறப்பது (`ngOnInit`) - அடுப்பு பற்றவைத்து, பால் வாங்கி வைப்பது. நாள் முழுவதும் விலை மாறினால் பலகையைப் புதுப்பிப்பது (`ngOnChanges`).

இரவு கடையை மூடும்போது (`ngOnDestroy`) அடுப்பை அணைத்து, பாத்திரங்களைக் கழுவி, விளக்கை அணைக்க வேண்டும். இதைச் செய்யாவிட்டால் (unsubscribe, timer நிறுத்துதல்) மறுநாள் பிரச்சனை - அதுதான் memory leak.

PROS

- Clear points to start/stop work
- Prevents memory leaks when used correctly

CONS

- Many hooks to remember
- Wrong hook choice causes ExpressionChanged errors

ADVANCED / NEW FEATURES

- `afterNextRender` / `afterEveryRender`
- `DestroyRef.onDestroy`
- Effects often replace some hooks

Common mistakes

- Doing heavy work (like HTTP calls) in the constructor instead of `ngOnInit`.
- Forgetting to clean up intervals/subscriptions in `ngOnDestroy`.
- Accessing a child view in `ngOnInit`; it is only ready in `ngAfterViewInit`.

Interview questions

Q1. Name the common lifecycle hooks in order.
constructor, ngOnChanges, ngOnInit, ngDoCheck, ngAfterContentInit, ngAfterViewInit, ngOnDestroy.

Q2. Constructor vs ngOnInit?
The constructor only creates the object and injects dependencies; `ngOnInit` runs after inputs are set and is where setup belongs.

Q3. When does ngOnChanges run?
Before `ngOnInit` and whenever an input value changes.

Practice

1. Start a `setInterval` clock in `ngOnInit` and stop it in `ngOnDestroy`.
2. Add `ngDoCheck` to the logger and observe how often it runs.
3. Use `DestroyRef` (`inject(DestroyRef).onDestroy(...)`) instead of `ngOnDestroy`.

21. Content Projection and ViewChild

DEFINITION

Content projection lets a component accept HTML from its parent and place it in 'slots', using `<ng-content>`. It is how you build reusable wrappers such as cards, dialogs and tabs. Use the `select` attribute to create several named slots.

viewChild() gives a component a reference to an element or child component in its own template (marked with a template variable like `#box`). It is a signal, available once the view has rendered.

SYNTAX

```
<!-- card.component template -->
<ng-content select="[card-title]" />
<ng-content />                        <!-- default slot -->

<!-- using it -->
<app-card><h3 card-title>Hi</h3> body text</app-card>

box = viewChild.required<ElementRef<HTMLInputElement>>('box');   // <input #box>
```

EXAMPLE

```
import { Component, ElementRef, signal, viewChild } from '@angular/core';

// A reusable wrapper with three slots
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <header><ng-content select="[card-title]" /></header>
      <section><ng-content /></section>
      <footer><ng-content select="[card-footer]" /></footer>
    </div>
```

```
  `,
  styles: `.card { border: 1px solid #aaa; padding: 8px; border-radius: 6px; }`,
})
export class CardComponent {}

@Component({
  selector: 'app-root',
  imports: [CardComponent],
  template: `
    <app-card>
      <h3 card-title>Masala Dosa</h3>
      <p>Crispy dosa with potato filling.</p>
      <small card-footer>Price: ₹60</small>
    </app-card>

    <!-- #nameInput is a template reference variable used by viewChild -->
    <input #nameInput placeholder="Your name" />
    <button id="focus" (click)="focusInput()">Focus the box</button>
    <p>{{ status() }}</p>
  `,
})
export class AppComponent {
  nameInput = viewChild.required<ElementRef<HTMLInputElement>>('nameInput');
  status = signal('Not focused yet');

  focusInput() {
    const el = this.nameInput().nativeElement;
    el.focus();
    this.status.set(`Input focused: ${document.activeElement === el}`);
  }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

You buy an empty gift box: the shop gives the box, lid and ribbon, and you choose what to put inside. `<ng-content>` is the space for your items.

தமிழ்

கடையில் ஒரு காலிப் பரிசுப் பெட்டி வாங்குகிறீர்கள். பெட்டியின் வடிவம், மூடி, ரிப்பன் எல்லாம் கடைக்காரர் தருகிறார்; உள்ளே என்ன வைக்க வேண்டும் என்பது உங்கள் விருப்பம் - இனிப்போ, துணியோ, புத்தகமோ.

`<app-card>` அந்தப் பெட்டி; `<ng-content>` உள்ளே வைக்கும் இடங்கள் (மேலே தலைப்பு, நடுவில் பொருள், கீழே விலை). `viewChild` என்பது பெட்டிக்குள் இருக்கும் ஒரு குறிப்பிட்ட பொருளைத் தொட்டு எடுப்பது போல.

PROS

- Flexible wrapper components
- Multiple slots with `select`
- Keeps components generic

CONS

- Projected content is created even if not shown
- Harder to type-check than inputs

ADVANCED / NEW FEATURES

- Signal queries `viewChild()` / `contentChild()` (stable v19)
- `ngProjectAs`

Common mistakes

- Reading `viewChild()` in the constructor or `ngOnInit`; it is only available after the view is created.
- Expecting `<ng-content>` to render conditionally; projected content is always created by the parent.
- Using `document.getElementById` instead of `viewChild`.

Interview questions

Q1. What is content projection?
Passing template content from a parent into a child component's `<ng-content>` slots.

Q2. What is `viewChild`?
A query that returns a child element/component from the component's own template.

Q3. `viewChild` vs `contentChild`?
`viewChild` queries the component's own template; `contentChild` queries projected content passed in by the parent.

Practice

1. Build a `tabs` wrapper that projects two panels.
2. Create a modal wrapper with `header`, `body` and `footer` slots.
3. Use `viewChild` to scroll a `div` to the top when a button is clicked.

22. Custom Directives

DEFINITION

A **custom directive** adds reusable behaviour to any existing element, without a template of its own. You create one with `@Directive` and a CSS-attribute **selector** such as `[appHighlight]`.

The `host` property connects the directive to the element it sits on: `'(mouseenter)'` listens to events and `'[style.color]'` binds styles. Directives can also have `input()`s and `output()`s like components.

SYNTAX

```
@Directive({
  selector: '[appHighlight]',
  host: { '(mouseenter)': 'onEnter()', '[style.background-color]': 'color()' },
})
export class HighlightDirective {
  color = input('yellow', { alias: 'appHighlight' });
}

<li appHighlight="lightgreen">Idli</li>
```

EXAMPLE

```
import { Component, Directive, input, output, signal } from '@angular/core';
```

```ts
// An attribute directive: adds hover highlighting to ANY element
@Directive({
  selector: '[appHighlight]',
  host: {
    '(mouseenter)': 'enter()',
    '(mouseleave)': 'leave()',
    // style binding on the host element
    '[style.background-color]': 'active() ? color() : null',
  },
})
export class HighlightDirective {
  color = input('yellow', { alias: 'appHighlight' }); // <li appHighlight="lightgreen">
  hoverChange = output<boolean>();                     // tell the parent about hover
  active = signal(false);

  enter() {
    this.active.set(true);
    this.hoverChange.emit(true);
  }
  leave() {
    this.active.set(false);
    this.hoverChange.emit(false);
  }
}

@Component({
  selector: 'app-root',
  imports: [HighlightDirective],
  template: `
    <ul>
      @for (item of items; track item) {
        <li appHighlight="lightgreen" (hoverChange)="onHover(item, $event)">{{ item }}</li>
      }
    </ul>
    <p>Hovered: {{ hovered() || 'nothing' }}</p>
  `,
})
export class AppComponent {
  items = ['Idli', 'Filter Coffee', 'Vada'];
  hovered = signal('');

  onHover(item: string, isOver: boolean) {
    this.hovered.set(isOver ? item : '');
  }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A teacher uses a yellow highlighter on any book line. The text does not change; only the look does. An attribute directive works like that highlighter on any element.

தமிழ்

பள்ளியில் ஆசிரியர் ஒரு மஞ்சள் மார்க்கர் (highlighter) பேனாவை வைத்திருக்கிறார். எந்தப் புத்தகத்தின் எந்த வரியிலும் அதைப் பயன்படுத்தி முக்கியமான இடத்தை அடையாளம் காட்டலாம். வரியின் உள்ளடக்கம் மாறுவதில்லை; அதன் தோற்றம் மட்டும் மாறுகிறது.

Custom directive அந்த மார்க்கர் பேனா: `appHighlight` என்று எந்த element மீதும் எழுதிவிட்டால் போதும், அதில் மவுஸ் வந்தால் நிறம் மாறும். ஒரு முறை எழுதி எங்கும்

மறுபடி பயன்படுத்தலாம்.

PROS

- Reusable behaviour
- No template needed
- Composable with `hostDirectives`

CONS

- Can hide behaviour from template readers
- Over-engineering risk

ADVANCED / NEW FEATURES

- `host` metadata object
- Directive composition API
- Signal inputs for directives

Common mistakes

- Using a selector without brackets (`appHighlight` instead of `[appHighlight]`) so it matches an element name.
- Not prefixing the selector (use `app` or your own prefix) and clashing with HTML attributes.
- Manipulating `nativeElement.style` directly when a host binding would do.

Interview questions

Q1. What is a directive?
A class that adds behaviour to elements. Types: components, structural and attribute directives.

Q2. How is an attribute directive created?
With `@Directive({ selector: '[name]' })`, using `host` bindings/listeners or `inject(ElementRef)`.

Q3. What does `host` do?
It binds properties, attributes, styles and events on the element that hosts the directive.

Practice

1. Create `appAutoFocus` which focuses its element when the page loads.
2. Write `appUppercase` that converts typed text in an input to capital letters.
3. Build `appTooltip="text"` that shows the text as the element's `title` attribute.

Advanced Level

23. HTTP Interceptors and Error Handling

DEFINITION

An **interceptor** is a function that sees every HTTP request/response. Use it to add auth tokens, log, show a global loader or handle errors in one place. Register with `provideHttpClient(withInterceptors([...]))`.

EXAMPLE

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).token();
  return next(token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req);
};
provideHttpClient(withInterceptors([authInterceptor]))
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Every parcel leaving a courier office passes one counter where the company stamps it. An HTTP interceptor stamps every request (token, logging) in one place.

தமிழ்

ஒரு கூரியர் அலுவலகத்திலிருந்து வெளியே செல்லும் ஒவ்வொரு பார்சலும் ஒரே கவுண்டரில் முத்திரை பெறுகிறது. அதுபோல HTTP interceptor ஒவ்வொரு request-க்கும் token, log போன்றவற்றை ஒரே இடத்தில் சேர்க்கிறது.

PROS

- One place for cross-cutting HTTP logic
- Keeps services clean
- Easy global error handling

CONS

- Order of interceptors matters
- Can hide side effects

ADVANCED / NEW FEATURES

- Functional interceptors with `withInterceptors`
- Works with `withFetch()`

24. Advanced RxJS Operators

DEFINITION

Flatten inner Observables with `switchMap` (cancel old), `mergeMap` (parallel), `concatMap` (queue), `exhaustMap` (ignore new). Combine streams with `combineLatest` / `forkJoin`; tame events with `debounceTime` and `distinctUntilChanged`; recover with `catchError` / `retry`.

EXAMPLE

```
search.valueChanges.pipe(
  debounceTime(300), distinctUntilChanged(),
  switchMap(term => api.search(term).pipe(catchError(() => of([]))))
)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

In a search box, you only want the result for the latest word typed, not for each letter. `switchMap` cancels the old request, like a waiter forgetting the old order when you change it.

தமிழ்

தேடல் பெட்டியில் கடைசியாக தட்டச்சு செய்த சொல்லுக்கான முடிவு மட்டுமே தேவை. `switchMap` பழைய request-ஐ நிறுத்திவிடும் - ஆர்டரை மாற்றினால் சர்வர் பழைய ஆர்டரை மறந்துவிடுவது போல.

PROS

- Handles races and cancellation
- Concise async pipelines

CONS

- Choosing the wrong flattening operator causes bugs
- Hard to debug

ADVANCED / NEW FEATURES

- `debounce` and `resource` helpers with signals
- RxJS 7 tree-shakable imports

25. State Management (Signals, Services, NgRx)

DEFINITION

Start simple: a service holding `signal`s, exposing read-only signals and update methods (a *store*). When many features share complex state, use **NgRx** (actions, reducers, selectors, effects) or **NgRx SignalStore** for predictable, debuggable state.

EXAMPLE

```
@Injectable({ providedIn: 'root' })
export class TodoStore {
```

```
  private _todos = signal<Todo[]>([]);
  readonly todos = this._todos.asReadonly();
  readonly doneCount = computed(() => this._todos().filter(t => t.done).length);
  add(t: Todo) { this._todos.update(l => [...l, t]); }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop keeps one common stock register that billing, store room and owner all read. A signal store is that single register for the whole app.

தமிழ்

ஒரு கடையில் பில்லிங், கிடங்கு, உரிமையாளர் அனைவரும் பார்க்கும் ஒரே பொது இருப்புப் பதிவேடு இருக்கும். Signal store என்பது முழு செயலிக்கும் அந்த ஒரே பதிவேடு.

PROS

- Predictable data flow
- Easy debugging (NgRx DevTools)
- Simple signal stores need little code

CONS

- NgRx adds boilerplate
- Over-engineering small apps

ADVANCED / NEW FEATURES

- NgRx SignalStore
- `linkedSignal`
- `resource` API for async state

26. Change Detection and OnPush

DEFINITION

Change detection decides when to refresh templates. `ChangeDetectionStrategy.OnPush` re-checks a component only when an input reference changes, an event fires inside it, an async pipe/signal it reads updates, or you call `markForCheck()`. Combine OnPush + signals + immutable data for speed.

EXAMPLE

```
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, ... })
export class RowComponent { row = input.required<Row>(); }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A supervisor checks only the counters where something changed instead of walking through the whole mall every minute. OnPush checks only changed components.

தமிழ்

ஒவ்வொரு நிமிடமும் முழு வணிக வளாகத்தையும் சுற்றாமல், மாற்றம் நடந்த கவுண்டர்களை மட்டும் மேற்பார்வையாளர் பார்ப்பது போல OnPush மாற்றமுள்ள component-ஐ மட்டும் சரிபார்க்கும்.

PROS

- Big performance gains
- Predictable updates
- Works great with signals

CONS

- Mutating objects breaks updates
- Needs immutable thinking

ADVANCED / NEW FEATURES

- Zoneless change detection (stable, default in new v21 apps)
- Signals drive updates

27. Advanced Dependency Injection

DEFINITION

Use `InjectionToken` for non-class values (config, API URL), `useClass` / `useValue` / `useFactory` providers to swap implementations, and component-level `providers` to get one instance per component. Injectors form a hierarchy: element -> environment -> root.

EXAMPLE

```
export const API_URL = new InjectionToken<string>('API_URL');
providers: [{ provide: API_URL, useValue: 'https://api.example.com' }]
const url = inject(API_URL);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A school office can give the same form (token) to every class but each class may fill it with different rules (providers). Tokens let you swap implementations easily.

தமிழ்

பள்ளி அலுவலகம் எல்லா வகுப்புக்கும் ஒரே படிவத்தை (token) கொடுக்கும்; ஆனால் ஒவ்வொரு வகுப்பும் தன் விதிகளுடன் நிரப்பலாம். Token மூலம் செயலாக்கத்தை எளிதாக மாற்றலாம்.

PROS

- Swappable implementations

- Better testing
- Scoped instances

CONS

- Complex injector trees are hard to reason about

ADVANCED / NEW FEATURES

- `provideAppInitializer`
- `EnvironmentProviders`
- Simplified `inject()` options

28. Deferrable Views and Performance

DEFINITION

`@defer` lazy-loads part of a template (on viewport, idle, interaction, timer) with `@placeholder`, `@loading`, `@error` blocks. Other wins: `@for` with `track`, OnPush, lazy routes, `NgOptimizedImage`, pure pipes and bundle analysis.

EXAMPLE

```
@defer (on viewport) { <app-heavy-chart /> }
@placeholder { <p>Chart loads when visible</p> }
@loading { <p>Loading...</p> }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bakery bakes special cakes only when a customer actually orders them, not all day in advance. `@defer` loads heavy parts only when needed.

தமிழ்

பேக்கரி சிறப்பு கேக்குகளை வாடிக்கையாளர் ஆர்டர் செய்யும்போது மட்டுமே தயாரிக்கிறது. `@defer` கனமான பகுதிகளை தேவைப்படும்போது மட்டுமே ஏற்றுகிறது.

PROS

- Smaller initial bundle
- Declarative lazy loading
- Built-in placeholder/loading states

CONS

- Layout shift if placeholders are not sized
- Needs triggers designed carefully

ADVANCED / NEW FEATURES

- Incremental hydration (v19)
- New triggers: viewport, idle, interaction, hover, timer

29. Testing Angular Apps

DEFINITION

Unit-test services as plain classes, test components with `TestBed` and `ComponentFixture`, mock HTTP with `provideHttpClientTesting`, and run end-to-end tests with Playwright or Cypress. Angular 21 uses Vitest by default for new projects.

EXAMPLE

```
it('adds numbers', () => {
  const calc = new CalculatorService();
  expect(calc.add(2, 3)).toBe(5);
});
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Before a bus leaves, the mechanic checks brakes, lights and horn. Automated tests are that checklist run on every build.

தமிழ்

பேருந்து புறப்படும் முன் மெக்கானிக் பிரேக், விளக்கு, ஹாரன் ஆகியவற்றைச் சோதிக்கிறார். Automated tests ஒவ்வொரு build-க்கும் அந்தச் சோதனைப் பட்டியலை இயக்குகின்றன.

PROS

- Safe refactoring
- Catches regressions
- Documents behaviour

CONS

- Takes time to write
- Brittle tests if overly coupled to the DOM

ADVANCED / NEW FEATURES

- Vitest default in v21
- Zoneless testing support
- Jasmine/Karma deprecated

30. SSR, Hydration and SEO

DEFINITION

Server-Side Rendering renders HTML on the server for faster first paint and SEO; **hydration** reuses that HTML in the browser instead of rebuilding it. Add with `ng add @angular/ssr`. Guard browser-only APIs with `isPlatformBrowser` or `afterNextRender`.

EXAMPLE

```
ng add @angular/ssr
provideClientHydration(withEventReplay())
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A restaurant serves a ready-made plate (server rendered page) instead of making the guest cook; hydration is the guest only adding the final seasoning.

தமிழ்

உணவகம் விருந்தினரை சமைக்கச் சொல்லாமல் தயாரான தட்டை (server-rendered page) பரிமாறுகிறது; hydration என்பது விருந்தினர் கடைசி மசாலா மட்டும் சேர்ப்பது.

PROS

- Faster first paint
- Better SEO
- Better perceived performance

CONS

- More complex hosting
- Browser-only APIs need guards

ADVANCED / NEW FEATURES

- Incremental hydration
- Event replay
- Route-level render modes (SSR/SSG/CSR)
- Hybrid rendering

31. Animations and UI Libraries

DEFINITION

Use CSS transitions with `animate.enter` / `animate.leave` for enter-leave effects (the older `@angular/animations` package is deprecated). **Angular Material** and **CDK** provide accessible buttons, dialogs, tables, forms and overlays.

EXAMPLE

```
ng add @angular/material
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Temple lamps fade softly instead of switching on suddenly. Animations make UI changes feel smooth.

தமிழ்

கோயில் தீபங்கள் திடீரென எரியாமல் மெதுவாக ஒளிர்கின்றன. Animation UI மாற்றங்களை மென்மையாக உணரச் செய்கிறது.

PROS

- Smoother UX
- CSS-based animations are fast

CONS

- Overuse distracts and hurts accessibility

ADVANCED / NEW FEATURES

- `animate.enter` / `animate.leave` (v20.2)
- `@angular/animations` deprecated

Master Level

32. Real Project 1: Task Manager

DEFINITION

Build a task app with standalone components, a signal-based `TaskStore`, reactive forms (add task with validation), `@for` lists with filters (All/Active/Done), `computed` counts, routing (`/tasks`, `/tasks/:id`) and `localStorage` persistence through an `effect`.

EXAMPLE

```
ng new task-manager --style=css
cd task-manager && ng g c tasks/task-list && ng g s tasks/task-store
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop owner's daily to-do board: add job, mark done, filter pending. A task manager app follows the same flow.

தமிழ்

கடை உரிமையாளரின் தினசரி வேலைப் பலகை: வேலை சேர், முடிந்ததாகக் குறி, நிலுவையில் உள்ளவற்றை வடிகட்டு. Task manager app அதே வழியில் செயல்படும்.

PROS

- Practises components, forms, routing, signals together

CONS

- Needs planning to avoid messy structure

ADVANCED / NEW FEATURES

- Use standalone components, signals and zoneless for a modern setup

33. Real Project 2: Product Catalog with API

DEFINITION

Create a shop front-end: product list from a REST API (`HttpClient` + service), search with `debounceTime` + `switchMap`, product detail route, cart signal with total using `computed`, an auth interceptor and lazy loaded `/admin`. Deploy to Netlify, Firebase Hosting or Azure Static Web Apps.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

An online grocery store lists products from a server, searches quickly and keeps a cart. This project practices API integration end to end.

தமிழ்

ஆன்லைன் மளிகைக் கடை சேவையகத்திலிருந்து பொருட்களைப் பட்டியலிட்டு, விரைவாகத் தேடி, கூடை வைத்திருக்கும். இந்த project API இணைப்பை முழுமையாகப் பயிற்சி செய்ய உதவும்.

PROS

- Real-world API practice
- Covers search, cart, auth

CONS

- Needs a backend or mock API

ADVANCED / NEW FEATURES

- Use `httpResource` / `resource` and signal stores

34. Architecture and Best Practices

DEFINITION

Organise by **feature folders**, use smart (container) vs dumb (presentational) components, keep logic in services, use strict TypeScript and `strictTemplates`, prefer signals + OnPush, name consistently, lint with ESLint, format with Prettier, and keep components small.

EXAMPLE

```
src/app/
  core/      (singletons: auth, interceptors)
  shared/    (reusable UI, pipes)
  features/
    tasks/   (components, store, routes)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A hospital has separate wards, a common pharmacy and shared admin. Feature folders, shared and core modules mirror that layout.

தமிழ்

மருத்துவமனையில் தனித் தனி வார்டுகள், பொது மருந்தகம், பொது நிர்வாகம் இருப்பது போல feature, shared, core folders அமைக்கப்படுகின்றன.

PROS

- Easier onboarding

- Scales with team size
- Predictable structure

CONS

- Over-structuring small apps wastes time

ADVANCED / NEW FEATURES

- Standalone APIs remove NgModule boilerplate
- `ng generate` uses modern defaults

35. Security and Accessibility

DEFINITION

Angular sanitizes bindings to stop **XSS**; never bypass with `bypassSecurityTrust*` on user data. Store tokens carefully (prefer HttpOnly cookies), validate on the server, and enable CSP. Accessibility: semantic HTML, labels, keyboard focus, `aria-*`, colour contrast.

EXAMPLE

```
<button type="button" aria-label="Close dialog" (click)="close()">x</button>
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A jewellery shop checks every customer at the door and locks valuables, even when the staff is honest. Angular sanitises HTML by default, but you must still validate on the server.

தமிழ்

நகைக்கடை நேர்மையான ஊழியர்கள் இருந்தாலும் வாசலில் சோதனை செய்து நகைகளைப் பூட்டி வைக்கிறது. Angular HTML-ஐ தானாக சுத்தப்படுத்தினாலும் server-இல் சரிபார்ப்பு அவசியம்.

PROS

- XSS protection by default
- Built-in sanitisation
- CSP friendly

CONS

- Bypassing sanitisation re-opens risks

ADVANCED / NEW FEATURES

- Trusted Types support
- Stricter template checks

36. Build, Deploy and CI/CD

DEFINITION

`ng build` creates optimised files in `dist/`. Deploy to any static host (Nginx, Netlify, Vercel, Firebase, GitHub Pages, S3+CloudFront); serve `index.html` for unknown paths. Automate with CI: install, lint, test, build, deploy.

EXAMPLE

```
ng build --configuration production
npx http-server dist/my-app/browser -p 8080
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A caterer packs the finished food in boxes and delivers it to the venue. `ng build` packs your app; hosting delivers it to users.

தமிழ்

சமையல்காரர் தயாரான உணவைப் பெட்டிகளில் அடைத்து நிகழ்ச்சி இடத்துக்கு அனுப்புகிறார். `ng build` செயலியை தொகுக்கிறது; hosting அதைப் பயனர்களுக்கு வழங்குகிறது.

PROS

- Optimised, tree-shaken bundles
- Works on any static host
- Easy CI/CD

CONS

- SPA needs rewrite rules for deep links

ADVANCED / NEW FEATURES

- esbuild builder
- Prerendering/SSG
- Faster builds with caching

Interview Question Bank

Q1. What are standalone components?
Components that declare their own `imports` and need no NgModule; the default since Angular 19.

Q2. Signals vs RxJS?
Signals hold synchronous state with fine-grained updates; RxJS models asynchronous streams. Convert with `toSignal` / `toObservable`.

Q3. Explain change detection.
Angular checks bindings to update the DOM. Zone.js (or signals in zoneless mode) triggers it; OnPush limits checks to changed inputs/signals/events.

Q4. `switchMap` vs `mergeMap`?
`switchMap` cancels the previous inner request (search-as-you-type); `mergeMap` runs all in parallel.

Q5. How does lazy loading work?
Routes use `loadComponent` / `loadChildren` with dynamic `import()`, so the bundler creates a separate chunk downloaded on demand.

Q6. What is an injection token?
A key for providing and injecting values that are not classes.

Q7. Template-driven vs reactive forms?
Template-driven: simple, directive-based. Reactive: explicit model in TypeScript, typed, testable, scalable.

Q8. How do you optimise performance?
OnPush, signals, `@for` track, lazy loading, `@defer`, image optimisation, avoiding heavy template functions, bundle analysis.

Q9. How do you share data between unrelated components?
A shared service (signals or BehaviorSubject), router state, or a state library like NgRx.

Q10. What is hydration?
Reusing server-rendered DOM in the browser instead of re-rendering, after SSR.

Capstone Project Ideas

1. Task Manager with filters, forms and localStorage
2. Product catalog with search, cart and API integration
3. Blog with routing, lazy loaded admin and auth guard
4. Dashboard with charts, `@defer` and signal store
5. Real-time chat using WebSocket + RxJS